TOLL

Tolled **O**ne-way **L**anes for **L**ifelong pickup-and-delivery

Direction as a price, not a rule

Abstract

Lifelong multi-agent pickup and delivery (MAPD) in a warehouse is dominated by a single structural fact: the map has narrow aisles, and traffic in a narrow aisle interferes with itself. *Priority Inheritance with Backtracking* (PIBT) handles that interference locally and cheaply — a blocked robot lends its priority to the robot in its way and asks it to move — but it plans one timestep at a time and has no notion of an aisle at all. The obvious extension is to give aisles directions, as a warehouse floor manager would: make a contended corridor one-way while the traffic in it is one-way.

This document describes **TOLL** — *Tolled One-way Lanes for Lifelong pickup-and-delivery* — and its planner **TOLL-PIBT**, and reports what happened when it was measured. The name is the claim: a robot may drive the wrong way down a one-way aisle, and pays a toll for doing so. Its central design decision is negative and, we argue, general: a one-way rule must enter the planner as a **term in the objective**, never as a **constraint on the domain**. Deleting a counterflow move from a robot's candidate set — the natural implementation, and the one the original specification called for — destroys the property priority inheritance runs on, namely that a robot can always be pushed somewhere. Pricing the same move instead, at a cost above every other soft preference and below the value of a step of progress, preserves it. Measured this single distinction is worth between 1.9× and 3.7× throughput on every map where an aisle ever commits a direction, and is the largest effect in the system. Two further mechanisms follow from the same reasoning: an aisle signal needs a maximum green as well as a minimum, or a balanced warehouse starves half its traffic; and deadlock recovery must require a corroborated stall signal, or it spends its time dismantling ordinary queues.

We evaluate against two published lifelong baselines — Token Passing and RHCR — and against every ablation of the method itself, with bootstrap confidence intervals and permutation tests over five seeds. The result is scoped rather than universal, and we report it that way. Against both external baselines TOLL wins decisively and significantly on every map tested, at a twentieth to a three-hundredth of their cost per timestep. Against plain lifelong PIBT its aisle layer is ahead by 21–27% on the two aisle-shaped maps and behind by 18–19% on the two open ones — and at five seeds the losses are significant while the wins ($p = 0.056$ and 0.063) are not yet. The mechanisms above are, wherever their mechanism can act at all. The final sections say exactly where that boundary falls, and what it would take to settle the part that is still open.

Source, simulator and every number in this document: `https://github.com/sinayanaik/aisleflow`
Figures regenerate from `docs/data/`; worked examples regenerate from the code.

The repository and the Python package predate this name and still read `aisleflow` and `lda_pibt`; `TOLL` and `TOLL-PIBT` are the method and its planner throughout this document.

Contents

I The method **5**

1 The problem, stated precisely **5**
1.1 Environment and agents . . . 5
1.2 The task stream . . . 5
1.3 Objective . . . 5
1.4 The traffic regime, and why it decides how to read every number . . . 6
1.5 What makes this hard, in one paragraph . . . 6

2 Prior methods, and how each one fails here **7**
2.1 Optimal MAPF: CBS and its relatives . . . 7
2.2 PIBT: one timestep, decided locally . . . 7
2.3 Token Passing: reserve a path, or wait . . . 7
2.4 RHCR: rolling-horizon windowed CBS . . . 8
2.5 Directional and structural traffic control . . . 8

3 What TOLL-PIBT changes, and why it is different **8**
3.1 Direction as a term in the objective, not a constraint on the domain . . . 9
3.2 The aisle as a traffic signal: a maximum green, not only a minimum . . . 10
3.3 Recovery on corroborated stalls only . . . 11
3.4 Cost . . . 11
3.5 What this does not claim . . . 12

II The system, in full **12**

4 The two-layer architecture **12**

5 Graph and warehouse model **14**
5.1 Grid graph . . . 14
5.2 BFS distance maps . . . 14
5.3 Structural features . . . 14
5.4 Aisle segmentation . . . 14

6 Robots and tasks **15**
6.1 Robot state . . . 15
6.2 Task urgency and waiting . . . 16
6.3 Arrival processes . . . 16

7 Priority function **16**
7.1 Task-class priority . . . 17
7.2 Fairness horizon . . . 17
7.3 Tie-breaking and ordering . . . 18

8 Candidate scoring **18**
8.1 Proximity mode . . . 18
8.2 Smooth direction weight β_i . . . 18
8.3 Aisle-continuity weight γ_i . . . 18
8.4 Turning cost . . . 19
8.5 Progress . . . 19

8.6 The full candidate score . . . 19
8.7 Preferred route direction and hysteresis . . . 20

9 Congestion model **21**
9.1 Occupancy index . . . 21
9.2 Downstream lookahead . . . 21
9.3 Congestion mixture . . . 21
9.4 Route-level congestion . . . 22

10 Aisle management **22**
10.1 Requested direction . . . 22
10.2 Per-robot directional demand . . . 22
10.3 Aggregate demand and the decision rule . . . 23
10.4 Aisle state machine . . . 24
10.5 Reservations . . . 24
10.6 Movement legality . . . 25

11 The PIBT core **25**
11.1 Candidate set . . . 25
11.2 Rejection rules, applied in this order . . . 26
11.3 Conflict checks . . . 26
11.4 The recursion . . . 27
11.5 Complexity . . . 27

12 Routing **28**

13 Task assignment **28**
13.1 Directional delay . . . 28
13.2 Blocking estimate . . . 28
13.3 The assignment cost . . . 28
13.4 Greedy matching . . . 29

14 Deadlock detection and recovery **30**
14.1 Three distinct stall signals . . . 30
14.2 Progress signal . . . 30
14.3 Repeated configurations and the wait-for graph . . . 30
14.4 Seven-level progressive recovery . . . 31

15 The lifelong simulation loop **31**

16 Evaluation metrics **32**
16.1 Percentile . . . 32
16.2 Jain's fairness index . . . 33
16.3 Aggregate report fields . . . 33

17 Statistics **33**
17.1 Percentile bootstrap confidence interval . . . 34
17.2 Permutation test . . . 34

18 Baseline algorithms **34**
18.1 Time-expanded A* . . . 35
18.2 Token Passing . . . 35
18.3 RHCR . . . 35

19 Parameter reference **36**
19.1 Defaults by section . . . 36
19.2 The ablation ladder and the factorial designs . . . 38

20 Glossary of symbols **39**

III What the measurements said **39**

21 Results **40**
21.1 Setup . . . 40
21.2 Against the published baselines . . . 40
21.3 Saturation, in one picture . . . 43
21.4 Which mechanisms earn their cost . . . 44
21.5 The ablation ladder is not monotone . . . 45
21.6 Hypotheses . . . 46
21.7 De-confounded findings . . . 47
21.8 The honest headline . . . 47
21.9 Reading these numbers . . . 48
21.10Threats to validity . . . 48

Part I
The method

1 The problem, stated precisely

> *A warehouse floor, a never-ending stream of jobs, and a fleet that must keep moving forever. This section fixes the model, the constraints and — the part that decides how every later number should be read — the traffic regime the measurements are taken in.*

1.1 Environment and agents

The warehouse is a 4-connected grid graph $G = (V, E)$. A vertex is a cell $v = (r, c)$; V holds the passable cells and E joins orthogonally adjacent ones. Shelving, walls and racking are simply absent from V. Certain vertices carry a role: $V_{\mathrm{p}} \subset V$ are pickup stations, $V_{\mathrm{d}} \subset V$ delivery stations, and $V_{\mathrm{k}} \subset V$ parking bays.

A fleet $A = \{1, \dots, n\}$ of identical robots occupies distinct vertices. Time is discrete and synchronous: at each timestep every robot simultaneously either moves to a vertex adjacent to its current one or stays where it is,

$$x_i(t+1) \in N(x_i(t)) \cup \{x_i(t)\}, \tag{1}$$

and a joint move is legal exactly when it creates neither of the two standard conflicts:

$$\text{vertex conflict:} \qquad x_i(t+1) = x_j(t+1), \qquad i \neq j, \tag{2}$$

$$\text{swap conflict:} \qquad x_i(t+1) = x_j(t) \ \wedge \ x_j(t+1) = x_i(t), \qquad i \neq j. \tag{3}$$

There is no rotation cost and no kinematic model; a robot's heading is tracked only so that changing it can be discouraged (Section 8.4).

1.2 The task stream

This is a *lifelong* problem, not a one-shot one. Tasks are not known in advance and never stop arriving. A task $\tau = (p, d, t_{\mathrm{rel}})$ releases at time t_{rel} and requires some robot to visit its pickup $p \in V_{\mathrm{p}}$ and then its delivery $d \in V_{\mathrm{d}}$, in that order. Arrivals follow one of four processes (Section 6.3); the experiments in this document use a Poisson process of mean rate λ tasks per timestep.

A robot is assigned at most one task at a time and is free again on delivery. Assignment is itself part of the problem — which robot takes which job matters — and is handled by a greedy minimum-cost match (Section 13) held *identical* across every planner compared in this document, so that comparisons isolate the movement layer.

The quantity a task contributes to the evaluation is its *service time*,

$$\mathrm{svc}(\tau) = t_{\mathrm{complete}}(\tau) - t_{\mathrm{rel}}(\tau), \tag{4}$$

defined only for tasks that actually complete — a restriction Section 1.4 returns to, because it is the single most misread number in the whole evaluation.

1.3 Objective

Over a horizon of T timesteps, with K tasks completed, the primary objective is **throughput**

$$\Theta = \frac{K}{T} \quad \text{(tasks per timestep)}, \tag{5}$$

subject to the joint plan being conflict-free at every step. Makespan — the standard objective for one-shot MAPF — is undefined here: there is no last task. Sum-of-costs is likewise unavailable, since the set of costs is not fixed in advance.

Secondary quantities are reported throughout, and each answers a different question: mean and p_{95} service time (latency, and tail latency), Jain's fairness index over per-robot completion counts (is the fleet used evenly), direction switches per 1000 steps (is the aisle signal stable), head-on conflicts and counterflow moves (is the aisle layer doing what it claims), and mean planner runtime per timestep (what the method costs).

1.4 The traffic regime, and why it decides how to read every number

The measurements in this document are taken in **saturation**, and this must be stated before any of them is quoted.

The arrival rate λ is set well above what the floor can serve. On `warehouse_medium` tasks arrive at $\lambda = 1.5$ per timestep and the best configuration completes about 0.50 per timestep; on `warehouse_bottleneck`, $\lambda = 0.8$ against 0.15 served. Between an eighth and a third of offered demand is served (Table 5); the backlog grows without bound for the whole run. Three consequences follow, and all three are routinely misread.

1. **Throughput measures capacity, not effort.** In saturation there is always another task waiting, so no planner is ever idle for want of work and Θ is exactly the floor's service capacity under that planner. This is why Θ is the primary objective and why differences in it are meaningful even when they look small: 0.50 against 0.31 tasks per timestep is a 61% difference in the capacity of the warehouse.
2. **Service time is censored, and the censoring is not random.** $\mathrm{svc}(\tau)$ exists only for completed tasks. A planner that serves only the nearby, easy tasks and abandons the rest reports an *excellent* mean service time. This is not hypothetical: on `warehouse_bottleneck` Token Passing reports a mean service time of 31 timesteps — against 154 for the planner that delivers twenty times as many tasks — because the handful of tasks it finishes are the ones it finishes in the first few timesteps, before the corridor jams for good. Mean service time may be compared only between planners with comparable throughput. Figure 2 shows this trap directly.
3. **Robot count is not a free parameter.** All planners are compared at the same fleet size on the same map, because throughput is not monotone in fleet size once the floor is congested — adding robots past the congestion knee lowers it.

> Every table in Section 21 therefore reports throughput first, quotes service time only alongside it, and never ranks two planners by service time alone. A reader who takes one thing from this section: *a low mean service time next to a low throughput is a symptom, not an achievement.*

1.5 What makes this hard, in one paragraph

A warehouse map is mostly corridors. In a single-file corridor, two robots wanting opposite directions cannot both proceed, and no local rule that treats them symmetrically resolves it; one must yield, and something must decide which. Worse, in a lifelong setting the pair that meets head-on is replaced by another pair a few timesteps later, so a resolution that works once must keep working forever, under an adversarial arrival process the planner does not control. Optimal joint planning would resolve this and is exponential (Section 2.1); planning each robot independently against a reservation table is cheap and deadlocks (Section 2.3). The interesting region is between the two, and that is where PIBT (Section 2.2) and this work sit.

2 Prior methods, and how each one fails here

> *Four families of solution, in increasing order of how much they give up for speed. Each is described by what it optimises, what one timestep costs it, and the specific way it breaks in a narrow lifelong warehouse — because that failure mode is what the next section is designed around.*

2.1 Optimal MAPF: CBS and its relatives

Conflict-Based Search (Sharon et al., 2015) solves one-shot MAPF optimally with a two-level search: a low level plans each agent independently, and a high level branches on the first conflict found between two agents' paths, adding a constraint forbidding one of them from the conflicting vertex or edge at that time and replanning only that agent. It is complete and optimal for sum-of-costs, and it is exponential in the number of conflicts — which in a dense warehouse corridor is very nearly the number of agent pairs.

Two properties disqualify it as-is here. First, cost: the high-level tree grows with contention, and contention is exactly the condition of interest. Second, and more fundamental, CBS answers a one-shot question. In a lifelong setting goals change continuously, so the plan CBS produces is stale before it is finished. Every practical lifelong system built on CBS therefore truncates it, which is what RHCR does.

2.2 PIBT: one timestep, decided locally

Priority Inheritance with Backtracking (Okumura et al., 2019) abandons multi-step planning entirely. At every timestep, robots are considered in descending priority order; each takes the best-scoring legal move available to it. If the cell it wants is occupied by a robot that has not yet decided, the requesting robot *lends its priority* to the occupant and asks it to move first — recursively. If the occupant can find no legal move, the request backtracks and the requester tries its next candidate. Section 11 gives the recursion in full.

Two properties matter for everything that follows.

1. **Cost.** One timestep is $O(|A|(\Delta(G) + F + \log|A|))$ for a fleet A, maximum degree $\Delta(G)$ and per-candidate scoring cost F. On the maps used here this is 0.4–1.0 ms per step, one to two orders of magnitude below either baseline in Section 21.
2. **The push property.** Priority inheritance works because a robot asked to move can, in general, be pushed somewhere. PIBT's progress guarantee is proved under the assumption that every edge of G lies on a simple cycle; informally, a robot displaced from a cell always has an alternative. This assumption is not a technicality — Section 3.1 shows that the natural way to implement one-way aisles *violates it by construction*, and that this, rather than anything about direction control as an idea, was the cause of every collapse previously attributed to the aisle layer.

What PIBT does not have is any notion of structure above a single cell. It cannot know that a corridor is a corridor, cannot commit a corridor to one direction, and cannot coordinate two robots that will meet in ten timesteps. Every robot's decision is local and one step deep. That is the gap this work addresses.

2.3 Token Passing: reserve a path, or wait

Token Passing (Ma et al., 2017) is the standard lifelong-MAPD baseline for exactly this pickup-and-delivery setting. A token circulates; a robot holding it plans a collision-free path to its goal through a shared reservation table using cooperative A* over (vertex, time) states,

commits that path into the table, and passes the token on. A robot that cannot find a path waits in place.

Its failure mode in a narrow warehouse is structural, not a matter of tuning: **Token Passing has no mechanism by which a blocked robot can displace the robot blocking it**. Once robots queue nose-to-tail in a single-file corridor, no robot in the queue can reserve a path through the robots ahead of it, every robot's search fails, every robot waits, and the configuration is absorbing — it never resolves for the rest of the run. On `warehouse_bottleneck`, whose two halves are joined by one corridor, all 16 robots reach this state and the run completes essentially no tasks. The first comparison animation in `docs/gifs/` shows precisely this, beside the same scenario under priority inheritance.

This is the failure PIBT was invented to remove, and it is worth being explicit that observing it here is a confirmation of the published analysis rather than a criticism of Token Passing: the algorithm was never claimed to be live in maps without alternative routes.

2.4 RHCR: rolling-horizon windowed CBS

Rolling-Horizon Collision Resolution (Li et al., 2021) makes CBS lifelong by bounding it in time. Every `replan_period` timesteps all robots are jointly replanned over a window of `window` timesteps using CBS restricted to that window; conflicts beyond the window's edge are simply not resolved yet, on the reasoning that they will be replanned before they arrive. Between replans, individual stale paths are repaired.

RHCR is the strongest of the three published methods for this setting and its design target is large: hundreds of robots on warehouses far bigger than the ones used here, where a rolling horizon pays for itself against full CBS. At the scale measured in this document, with untuned window parameters (10/5), it costs 43–98 ms per timestep — against 0.4–4.4 ms for the PIBT variants — and does not convert that cost into throughput. We report this as a property of *this baseline at this scale with these defaults* and not as a claim about RHCR generally; Section 21.10 states the caveat again where the numbers appear.

2.5 Directional and structural traffic control

Making corridors one-way is not a new idea; warehouse floors have been run that way by human traffic rules for as long as there have been warehouses, and highway-style directional layouts appear throughout the MAPF and AGV literature. The design decisions that distinguish proposals are: at what granularity a direction is decided (a whole map, a corridor, an edge), how often it may change, and — the question this work turns out to hinge on — **what a committed direction does to a move that opposes it**.

The usual answer is that it forbids it. A one-way corridor is one-way; a move against it is not legal; the planner never sees it. Under a planner that plans whole paths, this is harmless: the path simply routes around. Under PIBT it is not harmless at all, for the reason given in Section 2.2, and that is the subject of the next section.

3 What TOLL-PIBT changes, and why it is different

> *Four changes, three of them consequences of one idea: a high-level traffic rule may reorder what a local planner considers, and must never remove it. Each is stated as an inequality or a property, with the measurement that tests it.*

The architecture is two layers, and the division between them is the whole design (Section 4). The upper layer decides what each robot *wants*: which task, which waypoint, which route, which direction its aisle should be flowing, who plans first. The lower layer — PIBT, unmodified

— decides what is *allowed*. The upper layer may rank the lower layer's options. It may not delete them. Everything below follows from taking that sentence literally.

3.1 Direction as a term in the objective, not a constraint on the domain

3.1.1 The two implementations

Let $C_i = N(x_i(t)) \cup \{x_i(t)\}$ be robot i's candidate set and let $\text{safe}_i(v)$ hold when moving to v creates neither conflict of Equations (2) and (3). Write $\text{opp}_i(v)$ for the predicate "moving to v opposes the committed direction of the aisle being entered" (Section 10.6). The two ways to enforce a one-way rule are:

$$F_i^{\text{hard}} = \{\, v \in C_i \;:\; \text{safe}_i(v) \wedge \neg\, \text{opp}_i(v) \,\}, \tag{6}$$

$$F_i^{\text{soft}} = \{\, v \in C_i \;:\; \text{safe}_i(v) \,\}, \qquad S_i(v) \;-\!= \; \zeta \cdot \mathbf{1}\big[\text{opp}_i(v)\big]\,. \tag{7}$$

Equation (6) is what the original specification for this system called for, and what a one-way rule ordinarily means. Equation (7) is what TOLL-PIBT does: the move stays in the set, sorted to the back, and is taken when nothing better is available.

3.1.2 Why the difference is not cosmetic

Under a path planner the two are nearly equivalent — a forbidden edge simply raises the cost of the route through it, and the planner routes around. Under PIBT they are not, because PIBT's mechanism for resolving a blockage is to *push*, and pushing needs somewhere to push to.

Proposition 1 (Candidate preservation)**.** For every robot i and timestep t, $F_i^{\text{hard}} \subseteq F_i^{\text{soft}}$, and F_i^{soft} is exactly the set PIBT would have without any aisle layer at all. Consequently every joint move reachable under Equation (6) is reachable under Equation (7), and the aisle layer under Equation (7) cannot reduce the set of configurations the planner can escape from.

The converse fails, and it fails exactly where it hurts. Consider a single-file aisle $a = (u_1, \ldots, u_m)$ committed FORWARD (increasing index), occupied by robots $i_1, \ldots, i_m$, one per cell, with u_m's forward neighbour occupied by a robot that cannot move this timestep. Under Equation (6), each i_k has exactly two candidates: u_{k+1}, which is occupied, and staying put; the reverse move u_{k-1} has been deleted. Priority inheritance therefore walks the chain $i_1 \to i_2 \to \ldots \to i_m$, finds no robot with a free legal move, backtracks all the way, and every robot stays. The configuration is unchanged next timestep, and the timestep after that. Under Equation (7), i_1 retains u_0 at a cost of ζ, the chain terminates by pushing one robot backwards out of the aisle, and the aisle drains.

This is not a hypothetical. Instrumented on `warehouse_corridors` under Equation (6): all 35 robots stalled by $t = 120$, four aisles locked REVERSE, the identical global state still present at $t = 199$. The second comparison animation in `docs/gifs/` is this pair of runs side by side.

3.1.3 Sizing ζ

Pricing only works if the price is right. The candidate score (Section 8.6) is

$$\begin{aligned} S_i(v) = \underbrace{\alpha \cdot \text{progress}}_{\alpha=10} + \underbrace{\beta_i \cdot \mathbf{1}[\text{preferred direction}]}_{\beta \le 3} + \underbrace{\gamma_i \cdot \mathbf{1}[\text{stays in aisle}]}_{\gamma \le 2} - \underbrace{\lambda_{\text{turn}} P_{\text{turn}}}_{\le 1} \\ - \underbrace{\mu C_i(v)}_{\le 1} - \underbrace{\nu P_{\text{wait}}}_{\le 0.2} - \underbrace{\xi \mathbf{1}[\text{bottleneck}]}_{\le 1} - \underbrace{\zeta \mathbf{1}\big[\text{opp}_i(v)\big]}_{\zeta = 8}, \end{aligned} \tag{8}$$

and ζ is placed by two inequalities.

$\zeta < \alpha$**: progress can still buy its way through.** A counterflow move that closes one step of distance scores $\alpha - \zeta = +2$ before minor terms; staying in place scores $-\nu = -0.2$. So a robot in a corridor whose only progressing move is against the flow *takes it*, and pays. This is the inequality that makes Proposition 1 operational rather than merely set-theoretic: the retained candidate is not just present, it is reachable.

ζ **above every other preference: direction still governs.** Individually, $\zeta = 8$ exceeds the largest competing soft term ($\beta_{\max} = 3$) by 2.7×, and exceeds the sum of both positive preference terms ($\beta + \gamma = 5$). It does not exceed the arithmetic sum of all six non-progress terms at their simultaneous extremes (8.2), and we state that plainly rather than claiming a domination that the numbers do not support: the guarantee is that no ordinary combination of preferences reverses a direction decision, not that no contrived one can. What is guaranteed exactly is the ordering against progress, above.

3.1.4 What it is worth

Single-factor comparisons (`config.PAIRED_DESIGNS`), same seeds, same maps, the same direction decisions, differing only in Equation (6) versus Equation (7): 1.9× throughput on `warehouse_corridors`, 2.5× on `warehouse_medium` and 3.7× on `warehouse_narrow`, each at $p = 0.008$ — the floor of a five-seed permutation test — and up to 4.6× with entry admission also enabled. On `warehouse_bottleneck` the two arms are identical, because no aisle there ever commits a direction and the treatment has nothing to change.

It is the largest single effect measured anywhere in this system. Figure 5 plots it beside every other mechanism, and Section 21.4 gives the table.

> The general form of this finding is worth stating separately from TOLL-PIBT, because it applies to any local, push-based planner: *a traffic rule imposed as a hard constraint removes the freedom that local conflict resolution runs on.* If the planner resolves blockages by displacement, every constraint that shrinks the candidate set shrinks the set of blockages it can resolve. Price the rule instead, above the preferences and below the objective.

3.2 The aisle as a traffic signal: a maximum green, not only a minimum

An aisle decides its direction from the aggregate demand of the robots routed through it, S_a^+ forward and S_a^- reverse (Section 10.2), and commits when the imbalance leaves a dead band:

$$\text{commit FORWARD if } S_a^+ - S_a^- > \tau_{\text{switch}}, \qquad \text{REVERSE if } S_a^+ - S_a^- < -\tau_{\text{switch}}, \tag{9}$$

with a minimum lock $T_{\min}$ after each commit. Hysteresis of this kind is standard and it is only half a traffic signal: the dead band and the minimum lock bound how *soon* a direction may change, and nothing bounds how long it may persist.

That gap is not a corner case in a warehouse; it is the normal operating condition. Put pickups down one side of the floor and deliveries down the other — the standard layout, and three of the four maps used here — and loaded robots want one direction while empty robots want the other, in near-equal measure. The imbalance sits inside the dead band forever, and an aisle that committed a direction once holds it for the rest of the run while half its traffic waits.

TOLL-PIBT adds a maximum green $T_{\max}$: past $T_{\max}$ steps in one direction, *any* opposing demand at all forces the aisle to drain and flip. This gives a bounded-wait property that the dead band alone does not.

Proposition 2 (Bounded direction wait)**.** If an aisle a holds direction δ from time t_0 and there is opposing demand at every step, then a commits $\bar{\delta}$ no later than $t_0 + T_{\max} + T_{\text{drain}}$,

where $T_{\text{drain}} \leq$ `max_drain_time` is bounded because a DRAINING aisle that has not emptied within that many steps is force-reopened (Section 10.4).

Flips forced this way are counted separately as `starvation_flips`, which keeps two questions apart that a single switch counter conflates: "does hysteresis stop oscillation?" and "does anybody starve?"

Measured, the maximum green is *not* significant on throughput — and that is itself the finding. It was decisive while direction was a hard constraint, because then a starved robot had no way out at all. Once counterflow is merely expensive, a robot can buy its way out of a starved aisle, so liveness no longer rests on the signal alone: the two fixes are partly redundant. It stays enabled because it makes the aisle layer *correct* rather than merely survivable, and because its effect on switching is significant: on `warehouse_narrow`, 22.5 direction switches per 1000 steps against 1.0 without it ($p = 0.016$), of which 5.2 per run are flips the rule forced rather than demand. Without it, those aisles never turn at all. The third comparison animation shows an aisle that never flips beside one that must.

3.3 Recovery on corroborated stalls only

A deadlock detector in a lifelong warehouse faces an unusual difficulty: in saturation, the observable signature of a deadlock is also the observable signature of a perfectly healthy queue. Three stall signals are available (Section 14.1):

1. no route progress for t_{blocked} steps;
2. a cycle in the wait-for graph, where $i \to j$ when i's priority-inheritance request went to j;
3. a repeated configuration — the robot's last k positions equal the k before them.

Signal 1 alone describes every robot in every queue on a busy floor. Treating it as sufficient means recovery fires constantly on healthy traffic, and recovery's upper levels are drastic: temporary reverse movement, escape-vertex assignment, waypoint hijack (Section 14.4). Queues get taken apart and rebuilt continuously.

TOLL-PIBT requires corroboration: a group must show signal 1 *and* at least one of signals 2 or 3 before escalation begins. Signals 2 and 3 are the ones with a genuine deadlock semantics — a circular wait, or a state the system has demonstrably returned to. On `warehouse_corridors` this is worth 0.134 against 0.022 tasks per timestep ($p = 0.008$), a six-fold difference obtained entirely by declining to act on the weakest evidence. On the other maps it points the same way without clearing five-seed significance. The fourth comparison animation shows both.

There is a second-order lesson in how this defect stayed hidden. While direction was a hard constraint, the architecture was manufacturing real deadlocks continuously, so an over-eager detector looked useful: it *was* rescuing real deadlocks, of the system's own making. Fixing Section 3.1 is what made the over-eagerness visible.

3.4 Cost

The aisle layer is a constant-factor addition to PIBT, not a change in its complexity class. Per timestep, for fleet A on graph G with aisle set $\mathcal{A}$:

Planner	Per-timestep cost	Measured (ms/step)
PIBT (Okumura et al., 2019)	$O(\|A\|(\Delta(G) + F + \log \|A\|))$	0.4–1.0
TOLL-PIBT (this work)	$+ O(\|A\| + \|\mathcal{A}\|)$ aisle bookkeeping	1.4–4.4
Token Passing (Ma et al., 2017)	$O(\|A\| \cdot \mathrm{A}^*(\|V\| \cdot T_{\text{horizon}}))$	32–222
RHCR (Li et al., 2021)	windowed CBS, exponential in conflicts, capped	43–98

Table 1: Per-timestep cost. Measured columns are means over five seeds on the three baseline maps of Section 21; the spread is across maps and fleet sizes. The aisle layer costs a small constant factor over PIBT; both path-planning baselines cost one to two orders of magnitude more.

The added work per timestep is: updating each aisle's demand sums (one pass over robots), running the direction decision for each aisle (constant work each), granting reservations in priority order (one pass), and the deadlock detector's wait-for cycle search (linear in the fleet). None of it involves a search, and none of it is repeated per candidate — the congestion and distance terms consumed by the score are precomputed or cached (Sections 5.2 and 9), which is what keeps F at $O(1)$.

3.5 What this does not claim

The honest summary, stated here rather than buried in Section 21: the aisle layer is ahead where aisles are scarce and contended, and behind where they are not. Against plain lifelong PIBT it gains 27% on `warehouse_bottleneck` and 21% on `warehouse_corridors`, and loses 18% and 19% on the two open maps — and at five seeds *the two losses are significant and the two wins are not* ($p = 0.056$ and 0.063 against 0.016 and 0.008). The fifth comparison animation is the losing case, shown at the same size as the winning ones.

So the claim this document defends is precise: the mechanisms of this section are established, the scoping of the aisle layer to aisle-shaped maps is consistent and not yet significant, and Table 9 says which is which.

Part II
The system, in full

4 The two-layer architecture

> *Every mechanism in this document sits in one of two layers: an upper layer that decides what each robot* wants*, and a lower layer that decides what is* allowed*. Only the lower layer can prevent a collision, and nothing in the upper layer is permitted to remove a move the lower layer would have considered.*

TOLL-PIBT extends PIBT (Okumura et al., 2019) — a single-timestep, decentralised multi-agent collision-avoidance algorithm — with a high-level layer that decides *what* each robot wants (task, waypoint, preferred direction, priority) without ever touching collision safety.

HIGH LEVEL

`task stream` → `assignment` → `waypoints` → `routes` → `directional demand` → `aisle direction (hysteresis, drain)` → `reservations` → `priorities` → `deadlock detection & local recovery`

ranks and prices candidates; never removes one

LOW LEVEL (PIBT)

`candidates` → `safety rejection` → `score-sorted` → `priority inheritance` → `backtracking` → `one synchronised timestep`

The invariant this document keeps returning to: **everything above the line only reorders candidate moves; only the PIBT recursion (Section 11) and its validation companions (Section 11.3) decide collision-freedom.** No high-level formula can, by construction, produce two robots on the same vertex.

The middle row of that diagram is where this system differs from its own original specification, and from the usual way a directional rule is implemented. The specification listed "violates aisle direction" and "violates a reservation" among the low level's hard rejection rules; the same document's design principle said the high level never replaces PIBT's collision checks or its backtracking. Those two cannot both hold, and Section 3.1 works out which one had to give.

Table 2 maps the sections that follow onto the code.

Module	Section	What it does
`types.py`	—	enums, `Vertex`, `Reservation`
`config.py`	19	`Params`, ablation presets, factorial designs
`graph.py`	5	grid, BFS distance maps, articulation points, bridges
`warehouse.py`	5	map parsing, vertex metadata, aisle segmentation
`task.py`	6	tasks, queue, four arrival processes
`robot.py`	6	robot state
`congestion.py`	9	occupancy index, local / aisle / downstream congestion
`scoring.py`	8	proximity modes, weights, turning cost, $S_i(v)$
`priority.py`	7	task-class, waiting and blocked priority
`aisle_manager.py`	10	demand, hysteresis, drain-before-reverse, reservations
`assignment.py`	13	greedy $J(i,\tau)$ matching, waypoint transitions
`routing.py`	12	direction-aware A* (opt-in)
`pibt.py`	11	the PIBT recursion and its rejection rules
`deadlock.py`	14	wait-for cycles, repeated configurations, recovery
`validate.py`	11.3	vertex/swap validation, synchronised execution
`metrics.py`	16	throughput, service-time percentiles, Jain, runtime
`simulator.py`	15	the sixteen-step lifelong loop
`experiments.py`	21	ablation, factorial, baseline and hypothesis drivers
`baselines/`	18	Token Passing, RHCR, time-expanded A*
`stats.py`	17	bootstrap CI and permutation test

Table 2: Modules and the sections that describe them.

Notation. Throughout: i, j are robot indices and t the current timestep, advanced by exactly one per `Simulator.step()`. Vertices v, u, x are (row, col) pairs, and $x_i(t)$ is robot i's position at time t; g and w denote a goal and a waypoint. Greek letters are tunable weights, all fields of `config.Params`, collected in Section 19 and glossed in Section 20. $\mathbf{1}[\cdot]$ is the indicator function, and ∞ is the unreachable-distance sentinel.

5 Graph and warehouse model

> *The warehouse is a grid of open and blocked cells. This section defines how far apart two cells are, which cells are corridors, which are junctions, and which single cells the whole map depends on.*

5.1 Grid graph

`GridGraph` is a 4-connected grid: from $v = (r, c)$, the neighbour set is

$$N(v) = \{(r \pm 1, c),\ (r, c \pm 1)\} \cap V, \tag{10}$$

with the four deltas fixed in that order so that iteration is deterministic. Degree is $\deg(v) = |N(v)|$, and $\Delta(G) = \max_v \deg(v)$ (`GridGraph.max_degree`) appears in the PIBT complexity bound (Section 11.5).

5.2 BFS distance maps

For a goal g, `compute_bfs_distance_map(g)` runs a reverse BFS from g and returns, for every vertex v,

$$D_g(v) = d_{\text{route}}(v, g) \qquad \text{(hop count; } \infty \text{ if unreachable)}, \tag{11}$$

in $O(|V|)$ since the grid is unweighted. The map is cached per goal and is the single most reused primitive in the codebase: it backs route planning, progress scoring, congestion lookahead, task-assignment cost, and the A* heuristic for both the direction-aware router (Section 12) and the baseline planners (Section 18) — all without recomputation. `Warehouse.precompute()` eagerly builds D_g for every pickup, delivery and parking vertex.

`shortest_route(source, goal, horizon)` walks D_g greedily: at each step move to the neighbour n minimising $D_g(n)$, ties broken by the fixed neighbour order, so the result is deterministic. An optional `horizon` truncates the route after that many steps, used when only a lookahead prefix is needed (Section 9.2).

5.3 Structural features

Computed once by an iterative Hopcroft–Tarjan lowlink DFS (`_compute_articulation_and_bridges`, non-recursive to avoid Python's recursion limit on large maps):

- **Articulation points**: vertices whose removal disconnects G.
- **Bridges**: edges whose removal disconnects G — equivalently, edges lying on no cycle.
- **Dead ends**: $\{v : \deg(v) \leq 1\}$.
- **Intersections**: $\{v : \deg(v) \geq 3\}$ — exactly the vertices that separate one aisle from the next (Section 5.4).

`satisfies_pibt_reachability()` returns (dead ends $= \emptyset$) $\wedge$ (bridges $= \emptyset$), a rough check of PIBT's sufficient condition that every edge lie on a simple cycle — the condition its progress guarantee assumes, and the one Section 3.1 shows a hard one-way rule violates.

`bfs_ball(center, radius)` returns every vertex within `radius` hops of a centre, used for deadlock clustering (Section 14.3) and escape-vertex search (Section 14.4).

5.4 Aisle segmentation

An **aisle** is a maximal straight run of the graph after removing all intersection vertices — the corridors *between* junctions. Each aisle's `vertices` list is ordered from one endpoint to the

other (`_order_component`: find a local-degree-≤ 1 endpoint, then walk neighbour to neighbour). Moving to a higher index is FORWARD, to a lower index REVERSE, and

$$\text{entry\_direction}(v) = \begin{cases} \text{FORWARD} & \text{if index}(v) \leq (\ell - 1)/2, \\ \text{REVERSE} & \text{otherwise,} \end{cases} \tag{12}$$

for an aisle of length ℓ: entering nearer the start means heading forward through it.

> **Segmentation cuts at every turn.** Segmenting by connected component alone — the literal reading — produces L-, U- and T-shaped "aisles": `warehouse_corridors` yielded two 25-cell U shapes each spanning a whole corridor plus both vertical links. FORWARD has no single compass meaning on such a shape, and a one-way rule blocks its corner in both senses. `Aisle.axis` reports `row` or `col`, and is empty only for a single-cell aisle.

Capacity. `_aisle_capacity(`ℓ`)` has two models (`aisle_capacity_model`):

$$\texttt{"length"}: \quad \text{cap} = \text{clamp}(\lceil \rho \ell \rceil,\ 1,\ \texttt{aisle\_capacity}), \tag{13}$$

$$\texttt{"throughput"}: \quad \text{cap} = \text{clamp}(\lceil \rho \ell / T_{\min} \rceil,\ 1,\ \texttt{aisle\_capacity}), \tag{14}$$

with $\rho =$ `aisle_capacity_ratio` (default 1.0) and $T_{\min} =$ `minimum_aisle_lock_time` (default 20). The rationale for the second: a single-file corridor is throttled by how fast the robot at its *exit* can leave, not by how many robots physically fit inside. Packing it to ℓ just builds a queue that cannot drain.

Each aisle's minimum lock is $\max(2, \min(T_{\min}, 2\ell))$, capped so a short aisle does not lock disproportionately long for its own size.

Manageability. An aisle is permanently OPEN — never given a direction — if either $\ell <$ `directional_aisle_min_length` (default 4: too short for a rule to be worth anything) or some edge of the aisle, or either boundary edge to an adjacent intersection, is a bridge. One-waying a cut edge disconnects the graph, which no amount of pricing repairs.

6 Robots and tasks

> *What a robot remembers from one timestep to the next, and where the never-ending stream of jobs comes from.*

6.1 Robot state

`Robot` carries, among other fields: `priority` (recomputed every step, Section 7); `tie_breaker` $= 10^{-4} \cdot$ id, deterministic, unique, and small enough never to reorder two robots across a priority-class boundary; three related but distinct stall counters `waiting_time`, `blocked_time` and `no_progress_steps` (Section 14.1 distinguishes them precisely, because they feed different formulas); `route`, the current planned path; `route_distance_to_waypoint` $= D_w(x_i(t))$, the live BFS distance to the current waypoint; `mode`, a proximity bucket (Section 8.1); `direction_weight` and `aisle_weight` (β_i, γ_i, Sections 8.2 and 8.3); and `position_history`, a 16-entry deque used for repeated-configuration detection (Section 14.3).

6.2 Task urgency and waiting

$$\text{urgency}(t) = \begin{cases} 0 & \text{if deadline} = \infty, \\ \dfrac{1}{\max(1,\ \text{deadline} - t)} & \text{otherwise,} \end{cases} \tag{15}$$

$$\text{waiting\_time}(t) = \max(0,\ t - t_{\text{rel}}), \tag{16}$$

$$\text{service\_time} = t_{\text{complete}} - t_{\text{rel}} \quad \text{(defined only once completed).} \tag{17}$$

Urgency grows without bound as the deadline approaches and passes (slack $\to 0$ or negative), which is intentional: a task past its deadline should dominate the priority function's urgency term (Section 7) rather than saturate.

6.3 Arrival processes

Four modes, each returning a task count for `receive_new_tasks(t)`:

`periodic`: count = rate if t mod period $= 0$, else 0;
`poisson`: count $\sim$ Poisson(λ) via Knuth's sampler;
`bursty`: count = burst_size if t mod burst_period $= 0$, else 0;
`batch`: count = total if $t = 0$, else 0.

Knuth's Poisson sampler is the standard method for small-to-moderate λ:

Algorithm 1 Knuth's Poisson sampler (`task._poisson`)

Input: mean $\lambda > 0$
$L \leftarrow e^{-\lambda}$; $k \leftarrow 0$; $p \leftarrow 1$
loop
  $p \leftarrow p \cdot U$ ▷ $U \sim \text{Uniform}(0, 1)$
  if $p \leq L$ **then return** k
  end if
  $k \leftarrow k + 1$
end loop

After k multiplications $p = \prod_1^k U_j$, and the number of uniform draws needed to push that running product below $e^{-\lambda}$ is exactly Poisson distributed with mean λ (Knuth, 1997). The implementation guards at $k > 1000$ for numerical safety, which never triggers at realistic rates.

7 Priority function

> *Who plans first each timestep. Robots are ranked by what they are carrying, and a robot that has been waiting long enough always eventually outranks everyone — that bound is the fairness guarantee.*

The priority function must satisfy two competing goals: a loaded robot should almost always outrank a free one, so deliveries do not stall behind idle repositioning, and yet **no robot may starve forever**. TOLL-PIBT gets both from a class term plus an unbounded linear-in-waiting term:

$$p_i(t) = P_{\text{class}}(i) + k_w W_i + k_b B_i + k_u U_i + k_e E_i + \varepsilon_i, \tag{18}$$

Term	Meaning	Parameter
$P_{\text{class}}(i)$	task-class base priority (below)	`task_class_priority`
$k_w W_i$	waiting weight × `robot.waiting_time`	`waiting_weight` $= 5$
$k_b B_i$	blocked weight × `robot.blocked_time`	`blocked_weight` $= 10$
$k_u U_i$	urgency weight × task urgency (0 if no task)	`urgency_weight` $= 1$
$k_e E_i$	bonus if the robot is currently inside an aisle	`priority_inside_aisle` $= 50$
ε_i	tie-breaker, $10^{-4} \cdot$ id	—

7.1 Task-class priority

$$P_{\text{class}}(i) = \begin{cases} 400 & \text{state} = \text{RECOVERY}, \\ 300 & \text{task status} = \text{TO\_DELIVERY}, \\ 200 & \text{task status} = \text{TO\_PICKUP}, \\ 100 & \text{no task, state} = \text{PARKED}, \\ 0 & \text{otherwise.} \end{cases} \tag{19}$$

The ordering $P_{\text{emergency}} > P_{\text{loaded}} > P_{\text{pickup}} > P_{\text{repositioning}} > P_{\text{free}}$ says: a robot mid-recovery outranks everything; a robot carrying a delivery outranks one still travelling to a pickup; an idle robot heading for a parking bay slightly outranks a plain idle one, so it is not shoved off its chosen bay by another idle robot.

7.2 Fairness horizon

Because $k_w W_i$ grows without bound while P_{class} is fixed, any robot eventually outranks any other, however low its class. The time this takes is exact:

$$\text{fairness\_horizon} = \frac{P_{\text{emergency}} - P_{\text{free}}}{k_w} = \frac{400 - 0}{5.0} = 80 \text{ timesteps.} \tag{20}$$

After 80 steps of waiting, a P_{free} robot's waiting term alone exceeds the entire class spread, and it outranks even an emergency robot with $W = 0$. This is the guarantee `test_lifelong.py` checks directly. If $k_w \leq 0$ the function returns $+\infty$ — fairness would no longer be guaranteed, and it surfaces that rather than silently allowing it.

Worked example. The same scenario run out to timestep 149, so that robots have had time to accumulate waiting. Three of the 24: the highest-priority robot, the one that has waited longest, and the lowest-priority one.

robot	state	task	P_class	k_w·W	k_b·B	k_e·E	p_i(t)
robot 15	to_delivery	to_delivery	300	0	0	50	350.002
robot 3	to_pickup	to_pickup	200	15	0	50	265
robot 0	to_pickup	to_pickup	200	0	0	50	250

`P_class` sets the band; the waiting and blocked terms move a robot within it and, given long enough, out of it. The full class spread is 400 (400 for a robot in recovery down to 0 for an idle one) and `k_w` $= 5$, so 80 steps of waiting is worth the entire spread: a robot that waits that long outranks *any* robot of any class (Section 7.2).
That bound is the fairness guarantee, and it is the reason the priority function has a waiting term at all. Without it a permanently higher-class neighbour — a loaded robot on a floor that always has loaded robots — could starve an idle one indefinitely, and the throughput number would never show it.

7.3 Tie-breaking and ordering

`order_by_priority` sorts by $(-p_i, \text{id})$: descending priority, id as a deterministic secondary key. Combined with the ε_i term baked into p_i itself, ties are essentially never reached in practice, but the explicit id key guarantees determinism regardless.

8 Candidate scoring

> *Given the moves a robot is* allowed *to make, this is the single number that ranks them. Every preference in the system — direction, congestion, turning cost, aisle rules — is finally cashed out here, and nowhere else.*

Once PIBT (Section 11) has produced a *legal* candidate set, something must rank it: that ranking is what turns "collision-free" into "collision-free *and* purposeful". It is entirely separate from legality. Scoring only reorders candidates that already passed every rejection rule in Section 11.2.

8.1 Proximity mode

A robot's behaviour should change as it nears its target: far away, only progress and direction matter; up close, it should not zigzag chasing a one-step-shorter path. Route distance $r_i = D_w(x_i(t))$ is bucketed against $R_{\text{near}} = 2$ and $R_{\text{far}} = 8$:

$$\text{mode}(r_i) = \begin{cases} \textsc{transit} & r_i > R_{\text{far}}, \\ \textsc{approach} & R_{\text{near}} < r_i \le R_{\text{far}}, \\ \textsc{arrival} & r_i \le R_{\text{near}}. \end{cases} \tag{21}$$

8.2 Smooth direction weight β_i

Rather than a step function on the mode, the weight on "this move matched my preferred direction" decays smoothly as the robot approaches:

$$\beta_i = \beta_{\max} \cdot \min\left(1,\ \frac{\max(0,\ r_i - R_{\text{near}})}{\max(1,\ R_{\text{far}} - R_{\text{near}})}\right), \qquad \beta_{\max} = 3.0. \tag{22}$$

At $r_i \ge R_{\text{far}}$ it is at full strength; at $r_i \le R_{\text{near}}$ it is zero, leaving the robot free to approach from any angle. It is $\beta_{\max}$ unconditionally when $r_i = \infty$ (an unreachable route: keep whatever preference was already computed rather than collapsing it), and 0 when `direction_control` is `"none"`.

8.3 Aisle-continuity weight γ_i

A discrete analogue of the same idea, rewarding staying in the current aisle rather than cutting toward an intersection early:

$$\gamma_i = \begin{cases} 2.0 & \textsc{transit}, \\ 1.25 & \textsc{approach}, \\ 0.5 & \textsc{arrival}, \\ 0 & \texttt{direction\_control} = \texttt{"none"}. \end{cases} \tag{23}$$

8.4 Turning cost

$$P_{\text{turn}}(\text{prev}, \text{move}) = \begin{cases} 0 & \text{turning cost disabled, or either move is STAY,} \\ 0 & \text{move} = \text{prev} \quad \text{(straight on),} \\ \lambda_{\text{reverse}} = 2.0 & \text{move} = \overline{\text{prev}} \quad (180^\circ), \\ 1 & \text{otherwise} \quad (90^\circ). \end{cases} \tag{24}$$

8.5 Progress

The core "am I getting closer" signal, with two normalisations (`progress_normalization`). Writing $d_{\text{cur}} = D_w(x_i(t))$, $d_{\text{cand}} = D_w(v)$ and $\Delta = d_{\text{cur}} - d_{\text{cand}}$:

$$\text{progress} = \begin{cases} \Delta / \max(1, d_{\text{cur}}) & \texttt{"route"} \text{ (the literal specification),} \\ \Delta \in \{-1, 0, +1\} & \texttt{"step"} \text{ (the default).} \end{cases} \tag{25}$$

If the candidate is unreachable, progress is -1; if the current position is unreachable but the candidate is not, progress is 0 — neutral rather than an artificially huge gain.

> `"step"` is the default rather than the literal formula because at large d_{cur} the `"route"` normalisation shrinks $\alpha \cdot \text{progress}$ below $\beta_{\max}$ and λ_{turn}: at $d_{\text{cur}} = 20$, $\alpha \cdot \text{progress} \approx 10/20 = 0.5 < 3.0 = \beta_{\max}$. The dominance ordering the design claims (Section 8.6) silently inverts, and a distant robot prefers keeping its heading to closing distance.

8.6 The full candidate score

$$\begin{aligned} S_i(v) = \alpha \cdot \text{progress} \\ &+ \beta_i \cdot \mathbf{1}[\text{move} \neq \text{STAY} \wedge \text{move} = \text{preferred}] \\ &+ \gamma_i \cdot \mathbf{1}[v \text{ stays in the current aisle, robot not at an intersection}] \\ &- \lambda_{\text{turn}} P_{\text{turn}} - \mu\, C_i(v) - \nu\, P_{\text{wait}} - \xi \cdot \mathbf{1}[v \text{ is a bottleneck vertex}] \\ &- \zeta_{\text{counterflow}} \cdot \mathbf{1}[\text{move opposes the aisle's committed direction}] \\ &- \zeta_{\text{reservation}} \cdot \mathbf{1}[\text{entering a directional aisle without a ticket}]\,. \end{aligned} \tag{26}$$

Defaults: $\alpha = 10.0$, $\lambda_{\text{turn}} = 0.5$, $\mu = 1.0$, $\nu = 0.2$, $\xi = 1.0$, $\zeta_{\text{counterflow}} = \zeta_{\text{reservation}} = 8.0$. $P_{\text{wait}} = 1$ unless the candidate is "stay in place while already at the waypoint", which is free; otherwise waiting is always mildly discouraged, so a robot with any legal forward move prefers it.

The intended ordering is

$$\alpha \;>\; \zeta \;>\; \beta_{\max} \;>\; \gamma_{\max} \;>\; \lambda_{\text{turn}} \qquad (10 > 8 > 3 > 2 > 0.5)\,: \tag{27}$$

a whole step of progress beats everything; driving the wrong way down an aisle beats every other preference; a single turn is the cheapest term. Two conditions have to hold for that ordering to be real, and both were violated in earlier versions of this system.

$\zeta < \alpha$, and ζ a penalty rather than a rejection. The two ζ terms are where the aisle layer acts. Until this pass they were hard rejections in `feasible_candidates`, which deletes legal moves from the candidate set — and PIBT's progress argument rests on being able to push any robot into an adjacent cell (Section 11.4). Pricing counterflow keeps that freedom: a robot drives the wrong way when that is the only way to make progress, or when nothing else is left, and pays for it. `hard_direction_constraints` restores the old behaviour for comparison, and Section 3.1 is the full argument.

$\mu\, C_i(v) \le \mu$. $C_i(v)$ has to be normalised (Section 9.3) or the mixture is a robot *count*, and μC then reaches the scale of $\alpha\Delta$ — measured at mean 3.40, p_{90} 5.75, max 9.60 on `warehouse_medium` with 40 robots. That makes congestion the second-largest term in the score rather than a modulator of the smaller ones.

`CandidateScorer.sort_key(robot, v)` is $(-S_i(v), v)$: descending score, ties broken by vertex coordinates for determinism.

Worked example. `warehouse_medium`, variant `full_lda_pibt`, seed 7, 24 robots, timestep 18. Robot 3 is at (10,20) heading for waypoint (0,12), 18 steps away, so it is in `TRANSIT` mode with β_i = 3 and γ_i = 2 (Section 8.2, Section 8.3). Its previous move was `south` and its preferred direction is `north`. Every column below is computed by the same functions `CandidateScorer.score` itself calls, and they sum to the printed `S(v)`:

cell	move	α·prog	β·dir	γ·aisle	$-\lambda$·turn	$-\mu$·cong	$-\nu/-\xi$	$-\zeta$	S(v)	rejected by	priced
(9,20)	north	+10	+3	+0	−1	−0.11	+0	+0	11.89	vertex-conflict	-
(10,19)	west	+10	+0	+0	−0.5	−0.1	+0	+0	9.4	-	-
(10,20)	stay	+0	+0	+0	+0	−0.16	−0.2	+0	-0.36	-	-
(10,21)	east	−10	+0	+0	−0.5	−0.16	+0	+0	-10.66	-	-
(11,20)	south	−10	+0	+0	+0	−0.28	+0	−16	-26.28	-	aisle-direction, no-reservation

Read the last two columns first, because they are different kinds of thing. **Rejected by** is legality: those moves are gone before scoring, and no score could rescue them. **Priced** is preference: the move is still on the table, it just costs.
(9,20) scores highest at 11.89 and is unavailable regardless: `vertex-conflict` removed it in Section 11.2. So the move actually chosen is (10,19) at S = 9.4 — the best of what is *legal*.
The shape of the winning number is the whole argument: progress contributes +10, while every other term together moves it by −0.6. The preference terms break ties between moves that agree on progress; they never outvote progress itself. That is what α > ζ > β > γ > λ buys.
(11,20) is the instructive row. It pays 16 for `aisle-direction, no-reservation` and finishes 35.68 behind — last by a wide margin, and still in the candidate set. Set `hard_direction_constraints` and that same move is deleted instead; a move PIBT cannot see is a move priority inheritance cannot use to unblock a chain (Section 11.4). The gap between those two treatments is what the README's ablation measures as 1.9× to 3.1× throughput.

8.7 Preferred route direction and hysteresis

Algorithm 2 `compute_route_direction(robot)`

1: **if** |route| ≥ 2 **then return** the compass direction from route[0] to route[1]
2: **else return** the neighbour n maximising $D_w(x_i(t)) - D_w(n)$ ▷ steepest single-step descent
3: **end if**

`apply_direction_hysteresis` keeps the *previous* preferred direction unless one of: hysteresis is off; the previous preference was STAY; the proposed direction already matches; the robot has been blocked for at least t_{blocked} steps; the robot is not in TRANSIT mode; or the robot is at an intersection. In other words, **new direction commitments only happen at decision points**, which stops a robot flip-flopping mid-corridor as the greedy choice above wobbles.

9 Congestion model

> *How crowded a cell is about to be, expressed as one number between 0 and 1 so that it can modulate the score without ever dominating it.*

9.1 Occupancy index

`OccupancyIndex` is rebuilt every timestep in $O(n)$ for n robots: a vertex-to-robot hash map, an aisle-to-count counter, and a spatial hash with cell size $\max(1, R_{\text{local}})$ for fast local-density queries. Aisle load is a ratio, and may exceed 1:

$$\text{aisle\_load}(a) = \frac{\text{occupancy}(a)}{\max(1,\ \text{capacity}(a))}. \tag{28}$$

`local_density(v, exclude)` approximates $|\{j : d_1(v, x_j(t)) \le R_{\text{local}}\}|$, excluding one robot (typically the querying one), in two passes: sum the 3×3 bucket window around v for a cheap upper bound, and if it is nonzero, refine exactly by scanning the Manhattan diamond of radius $R_{\text{local}} = 3$.

9.2 Downstream lookahead

$$\text{downstream}(v, w) = \frac{1}{|\text{tail}|} \sum_{u \in \text{tail}} \mathbf{1}[u \text{ occupied}], \qquad \text{tail} = \text{shortest\_route}(v, w, K)[1{:}], \tag{29}$$

with $K =$ `downstream_horizon` $= 5$: the fraction of the next K route vertices beyond v currently holding a robot. It is cached per (v, w) per timestep, since many robots query the same downstream segment.

9.3 Congestion mixture

$$C_i(v) = \frac{\omega_{\text{loc}}\, \text{local\_ratio}(v) + \omega_{\text{aisle}}\, \text{aisle\_load}(a(v)) + \omega_{\text{down}}\, \text{downstream}(v, w_i)}{\omega_{\text{loc}} + \omega_{\text{aisle}} + \omega_{\text{down}}}. \tag{30}$$

All three weights default to 1.0, so $C_i(v)$ is the mean of three ratios and lies in $[0, 1]$. It returns 0 unconditionally when `congestion_scoring` is off.

The first term is a *ratio*, not a count:

$$\text{local\_ratio}(v) = \frac{\text{local\_density}(v)}{|\{u \in V : d_1(u, v) \le R_{\text{local}}\}|}, \tag{31}$$

the fraction of reachable cells within the radius that hold a robot. The other two were already ratios, so mixing a count with them made the local term effectively the whole mixture and let μC outrank the terms it is meant to modulate (Section 8.6). `congestion_normalisation =False` restores the raw-count form.

> Normalising is a *calibration* fix, not a performance one: on the bundled maps its measured effect on throughput is neutral to slightly negative. What it buys is that the weight ordering Equation (27) claims is the ordering the system actually has at runtime. That is worth stating plainly rather than presenting as a speed-up.

Worked example. The same robot and timestep. `C_i(v)` mixes three occupancy signals with weights ω`_local` $= 1$, ω`_aisle` $= 1$, ω`_down` $= 1$, then divides by their sum (3) because `congestion_normalisation` is on:

cell	C_local	C_aisle	C_down	C_i(v)	$-\mu \cdot$C

```
-------  -------  -------  ------  ------  ------
(9,20)   0.091    0.25     0       0.114   −0.114
(10,19)  0.091    0        0.2     0.097   −0.097
(10,20)  0.077    0        0.4     0.159   −0.159
(10,21)  0.091    0        0.4     0.164   −0.164
(11,20)  0.182    0.25     0.4     0.277   −0.277
```

Normalisation is what makes this a modulator rather than a rival. The largest `C_i(v)` here is 0.277, so the entire congestion term is worth at most 0.277 — against $\alpha = 10$ for a single step of progress. Turn normalisation off and `C_local` reverts to a raw robot count: μ·`C` then reaches the scale of $\alpha \cdot \Delta$ (measured mean 3.40, p90 5.75, max 9.60 on this map at 40 robots), and congestion stops modulating the smaller terms and starts overruling the largest one.

9.4 Route-level congestion

$$\text{route\_congestion}(s, g) = \frac{\text{occupied vertices on the route}}{|\text{route}|} + \frac{\sum_{a \in \mathcal{A}(\text{route})} \text{aisle\_load}(a)}{\max(1, |\mathcal{A}(\text{route})|)}, \quad (32)$$

where $\mathcal{A}(\text{route})$ is the set of distinct aisles the route touches. Used only by the assignment cost (Section 13), never by per-step candidate scoring.

10 Aisle management

> *Narrow corridors get a traffic light. Robots ask for a direction, the corridor decides from the aggregate, and it holds that decision long enough for the decision to be worth making — but not forever.*

The design principle, stated verbatim in the code twice: **robots generate directional requests, but aisles make directional decisions.** No single robot can force an aisle to flip; direction is a property the aisle decides from the aggregate of what is asking.

10.1 Requested direction

`requested_direction(robot, aisle)` projects the robot's planned route onto the aisle's own index ordering. If the route touches the aisle at two or more points, it is FORWARD when the index increases and REVERSE when it decreases; for a single touching vertex it falls back to `entry_direction` (Section 5.4).

10.2 Per-robot directional demand

$$\text{demand}_i(a) = w_u U_i + w_w W_i + w_p P_i - w_l L_i - w_c C_i, \quad (33)$$

Symbol	Definition	Meaning
U_i	task urgency, 0 without a task	deadline pressure
W_i	$\max(\text{waiting}, \text{no\_progress}) + \text{blocked}$	stalled progress counts as waiting
P_i	$1/(1 + \min(d(x_i, \text{start}), d(x_i, \text{end})))$	proximity to the nearer end
L_i	route distance to waypoint, 0 if ∞	long-route robots discounted
C_i	$\text{aisle\_load}(a)$	existing congestion discourages more demand

Weights: $w_u = 1.0$, $w_w = 0.5$, $w_p = 2.0$, $w_l = 0.05$, $w_c = 0.5$. The W_i term takes a maximum over two stall counters deliberately — a robot shuffling in place without gaining route distance is starving just as much as one standing still, and should be able to flip the aisle.

10.3 Aggregate demand and the decision rule

$$S_a^+ = \sum_{i \text{ requesting FORWARD}} \rho_i(a)\,\text{demand}_i(a), \quad S_a^- = \sum_{i \text{ requesting REVERSE}} \rho_i(a)\,\text{demand}_i(a), \tag{34}$$

$$\text{imbalance}(a) = S_a^+ - S_a^-. \tag{35}$$

$\rho_i(a)$ is the attribution weight. By default each robot is charged to exactly one aisle ($\rho = 1$ for its next aisle, else its current one, and 0 elsewhere). With `demand_spread` enabled the robot is charged to *every* aisle its route touches, with $\rho_i(a) = 1/(1 + \text{steps to the entry})$ — the literal reading of the specification, and the only one that lets an aisle see traffic more than one aisle away. Measured, it engages direction control considerably more often and costs throughput, so it ships off; the flag exists so the measurement stays reproducible.

`update_aisle_direction` then applies hysteresis with a dead band $[-\tau_{\text{switch}}, \tau_{\text{switch}}]$ ($\tau_{\text{switch}} = 5.0$ when hysteresis is on, else 0) *and* a maximum green:

Algorithm 3 `update_aisle_direction(a, t)`

if a is occupied and holds FORWARD **then**
  drain if imbalance $< -\tau_{\text{switch}}$ **or** $(t - \text{direction\_since} \geq T_{\max}$ **and** $S_a^- > 0)$
else if a is occupied and holds REVERSE **then**
  drain if imbalance $> +\tau_{\text{switch}}$ **or** $(t - \text{direction\_since} \geq T_{\max}$ **and** $S_a^+ > 0)$
else if a is empty and $t <$ lock_until **then**
  hold the current direction ▷ minimum-lock enforcement
else if a is empty and the lock has expired **then**
  commit FORWARD if imbalance $> \tau_{\text{switch}}$ and $S_a^+ > 0$
  commit REVERSE if imbalance $< -\tau_{\text{switch}}$ and $S_a^- > 0$
  otherwise **open** the aisle
end if

$T_{\max}$ = `maximum_aisle_lock_time` (default 40), floored at the aisle's own minimum lock so the two clocks cannot contradict each other.

Why a maximum green is necessary. Hysteresis is only half a traffic signal: it bounds how *soon* a committed direction may change and says nothing about how long it may persist. Consider a warehouse with pickups down one side and deliveries down the other — the standard layout, and three of the four bundled maps. Loaded robots want one direction and empty ones the other, in near-equal measure, so |imbalance| sits inside the dead band indefinitely and an aisle holds its first committed direction for the rest of the run while half its traffic never gets through. Instrumented on `warehouse_corridors` under a hard directional rule: all 35 robots stalled by $t = 120$, four aisles locked REVERSE with `lock_until` long expired, the identical state still present at $t = 199$. Proposition 2 states the property the fix restores.

Flips forced this way are counted separately as `starvation_flips`, which keeps "does hysteresis stop oscillation?" distinct from "does anybody starve?"

`_commit` increments `direction_switches` only on an *actual* change from a non-NONE previous direction, and sets lock_until $= t + T_{\min}$ and direction_since $= t$, so a freshly committed direction can neither be reversed for $T_{\min}$ steps nor held past $T_{\max}$ against opposing demand.

Worked example. Same run, timestep 18. Aisle 38 runs (11,20) -> (14,20) (length 4, capacity 4, axis `col`). Each robot routed through it contributes `w_u·U + w_w·W + w_p·P − w_l·L − w_c·C` (Section 10.2) to whichever direction it wants, and the two sides are summed:

```
S_a^+  (forward demand)      = -0.558
S_a^-  (reverse demand)      = 4.758
imbalance  S_a^+ − S_a^-     = −5.317
dead band  ± τ_switch        = 5
occupancy                    = 1 robot(s) inside
state on entry               = REVERSE, direction REVERSE
locked until t               = 26 (T_min = 8, T_max = 40)
```

With hysteresis on, `update_aisle_direction` returns `REVERSE`; with the dead band removed it returns `REVERSE`. The two agree here, which is the common case: hysteresis is not meant to fire often, it is meant to make the rare flip deliberate.
An imbalance of 5.317 against a dead band of 5 clears the band, and a committed direction is additionally locked for `T_min` $= 8$ steps. The maximum green matters for the opposite failure: past `T_max` $= 40$ steps *any* opposing demand forces a drain, so a balanced aisle - pickups on one side, deliveries on the other, imbalance permanently inside the band - cannot hold one direction forever and starve the traffic wanting the other way.

10.4 Aisle state machine

imbalance past $\pm\tau$, **or** held $\geq T_{\max}$ with opposing demand		
`forward`	$\longrightarrow$	`draining`
`reverse`	$\longrightarrow$	`draining`
`draining`	$\longrightarrow$	`reverse/forward` (occupancy reaches 0: commit pending)
`draining`	$\longrightarrow$	`open` (drain timed out)
`open`	$\longrightarrow$	`forward/reverse` (imbalance + lock expired)

Entering DRAINING records a `pending_direction`: the opposite of the current one. When the aisle empties, that direction is committed **directly**, without re-running the imbalance test. The demand that triggered the drain has usually moved on by the time the aisle is clear, so re-testing would simply re-commit the direction just drained, and the flip would never happen.

One addition beyond the base rule: a DRAINING aisle that has not reached zero occupancy within `max_drain_time` (default 30) steps is force-reopened. The specification allows an immediate switch when "the current direction is infeasible", and an aisle that cannot drain is exactly that. Without it the state machine has an absorbing deadlock.

10.4.1 Coordinated direction assignment

`coordinate_aisle_directions` (default off) decides directions as a *set* rather than one aisle at a time: aisles commit in descending |imbalance|, and any commit that would break strong connectivity of the directed residual graph is rolled back, leaving that aisle bidirectional. The check is a two-way BFS over passable cells under the current assignment, $O(|V| + |E|)$ per commit.

> This was the fix an earlier review expected to matter most. It does not: on the bundled maps its measured effect on throughput is between ± 0.000 and ± 0.011, because a single one-way aisle almost never disconnects a ladder graph, so the guard hardly ever fires. The collapse it was meant to cure was a liveness failure (Section 10.3), not a connectivity failure. It ships as a flag so the null result stays reproducible.

10.5 Reservations

`update_aisle_reservations` expires any reservation past its `expiry_time`, then grants a fresh one only if *all* of: the aisle is not DRAINING; the robot's requested direction matches the aisle's current one, or the aisle has none; the robot does not already hold one; occupancy plus

pending reservations held by robots still outside is below capacity; and the robot is within route distance 3 of either endpoint. A granted reservation carries

$$\text{expected\_exit} = t + \lfloor \ell \rfloor + 1, \qquad \text{expiry} = t + \texttt{reservation\_ttl}\ (= 15). \tag{36}$$

The read side, `can_enter_aisle`, splits into two gates because they answer different questions:

- the **occupancy cap** applies to every managed aisle, OPEN included. Over-filling a single-file corridor is what builds a queue that cannot drain, and that is true whether or not the corridor currently has a direction. This is the mechanism hypothesis H3 is about.
- the **ticket** is required only once the aisle is directional. Demanding one to enter an OPEN aisle throttles every crossing on a map whose aisles are mostly short and mostly open.

$$\text{can\_enter}(i, a) = \begin{cases} \text{false} & a \text{ is DRAINING}, \\ \text{false} & a \text{ is directional, direction mismatched}, \\ \text{false} & \text{occupancy}(a) \geq \text{capacity}(a), \\ \text{true} & \text{reservations off, or } a \text{ is OPEN}, \\ \text{has\_ticket}(i, a) & \text{otherwise.} \end{cases} \tag{37}$$

> **Gating.** These are gated on `reservations` alone. They used to be gated on `AisleManager.enabled` (that is, on `direction_control = "aisle"`), which made the flag a silent no-op in every robot-level configuration — including `reservations_only`, the variant that exists specifically to isolate it, whose runs were bit-identical to its own control. An entire factorial design was built on that empty cell (Section 21.7).

10.6 Movement legality

One deliberate deviation: direction restricts **entry**, not **interior movement**. A robot already inside a directional aisle may always make an *egress* move toward the nearer endpoint. With $c = \text{index(current)}$, $n = \text{index(candidate)}$ and ℓ the last index,

$$\text{is\_egress}(c, n) \iff \min(n, \ell - n) < \min(c, \ell - c), \tag{38}$$

that is, the candidate is strictly closer in index-distance to *whichever* endpoint is nearest, regardless of which one that is. Without this, a DRAINING aisle could trap a robot facing the "wrong" way, and the aisle would never empty.

11 The PIBT core

> *The only part of the system that decides collision-freedom. A robot takes the best move it can; if the cell it wants is occupied, it lends its priority to the occupant and asks it to move first.*

This is the safety-critical layer. Everything above only ranks and restricts candidates; only this recursion, with its validation companions (Section 11.3), decides what is collision-free.

11.1 Candidate set

$$C_i = N(x_i(t)) \cup \{x_i(t)\} \tag{39}$$

— every grid neighbour, plus staying in place. Staying is always in the set, which is what guarantees the recursion terminates with *some* move, possibly "stay put", never "no answer".

11.2 Rejection rules, applied in this order

1. **Off-graph**: the candidate is not in V.
2. **Kinematics**: currently a stub that always passes — a hook for future footprint or turning-radius constraints.
3. **Vertex conflict** (Section 11.3).
4. **Swap conflict with the calling parent**, checked only when this call is a recursive priority-inheritance request.

Surviving candidates are then sorted by score (Section 8.6, descending). Legality and preference are fully decoupled: scoring never overrides a rejection, and rejection never depends on score.

> **Aisle direction and reservations are deliberately not on this list.** Every rule here concerns safety or physical possibility; the aisle layer concerns preference, and acts through the two ζ terms of Equation (26) instead. The distinction is load-bearing rather than stylistic: Section 11.4's progress argument assumes a robot can always be pushed into an adjacent cell, and a one-way rule enforced by rejection removes exactly that. With aisle direction as a hard rejection, inheritance chains dead-end against the constraint and the aisle layer collapses — measured on `warehouse_narrow` at 0.078 tasks per step against 0.289 for the same configuration with the same direction decisions priced instead of enforced. Section 3.1 is the argument in full.
> `hard_direction_constraints=True` reinstates the old order (aisle direction, then reservation, before the conflict checks) and suppresses the ζ terms so the penalty is not counted twice.

11.3 Conflict checks

Vertex conflict, $O(1)$ via the reservation set:

$$\text{vertex\_conflict}(i, v) \iff v \in \mathcal{R}_t \ \vee \ \exists\, j \neq i \text{ at } v \text{ with } x_j(t+1) = v, \tag{40}$$

where $\mathcal{R}_t$ holds the vertices already tentatively claimed this step. Swap conflict, a two-cycle position exchange with the calling parent j:

$$\text{swap\_conflict}(i, j, v) \iff v = x_j(t) \ \wedge \ x_j(t+1) = x_i(t). \tag{41}$$

11.4 The recursion

Algorithm 4 `pibt(robot` i`, parent` j`,` t`)`

$F \leftarrow$ feasible_candidates(i, j, t) ▷ legality-filtered, score-sorted
for all $v \in F$ **do**
  if v is already reserved this step **then continue**
  end if
  tentatively set $x_i(t+1) \leftarrow v$ and reserve v
  $k \leftarrow$ the robot currently at v, if any
  if k is **none** or $k = i$ **then return** VALID ▷ empty cell, or staying put
  end if
  if $x_k(t+1)$ is already decided **then**
    if $x_k(t+1) \neq v$ **then return** VALID ▷ k is moving away; safe to follow
    else
      undo the reservation; $x_i(t+1) \leftarrow$ **none**; **continue**
    end if
  end if
  waiting_for$(i) \leftarrow k$ ▷ records the wait-for edge Section 14.3 reads
  if pibt$(k, i, t) =$ VALID **then return** VALID ▷ priority inheritance succeeded
  end if
  undo the reservation; $x_i(t+1) \leftarrow$ **none**; **continue** ▷ backtrack
end for
$x_i(t+1) \leftarrow x_i(t)$; reserve it **return** INVALID ▷ always legal, since $x_i(t) \in C_i$

`plan_step` calls pibt$(i, \textbf{none}, t)$ for every robot in descending priority order (Section 7), skipping any robot whose next position was already set by an earlier robot's recursive chain. This is the mechanism that makes priority meaningful: a low-priority robot occupying a high-priority robot's preferred cell is *asked* to move, and if it can find any legal alternative, both robots move without conflict. A chain of such requests is what "inheritance" refers to. An INVALID result — a robot forced to stay — increments `blocked_time`, which feeds back into both the priority function and deadlock detection.

Three statistics are tracked: `recursive_calls` (a recursion-depth proxy), `backtracks` (inheritance chains that failed and were undone), and `invalid_results` (robots genuinely blocked this step).

11.5 Complexity

One timestep costs

$$O(|A|\,(\Delta(G) + F + \log|A|))\,, \tag{42}$$

for fleet A, maximum graph degree $\Delta(G)$, and per-candidate scoring cost F — kept at $O(1)$ by the occupancy index, the spatial hash and the precomputed distance maps, so no candidate evaluation runs a BFS. The $\log|A|$ maintains priority order.

PIBT's theoretical progress guarantee — that some robot always has a legal move — assumes every edge lies on a simple cycle (Section 5.3). One-way aisle *constraints* remove that freedom locally, which is why deadlock recovery (Section 14) exists as a safety net. One-way aisle *prices* do not, which is why the safety net is needed far less often once Section 3.1 is applied.

12 Routing

> *How a robot works out a path to its next waypoint, and when that path is recomputed.*

By default `Router.route` is `graph.shortest_route` (Section 5.2) — plain BFS descent. When `direction_aware_routing` is enabled *and* `direction_control` is `"aisle"`, it instead runs a weighted A* using the BFS distance map as an admissible and consistent heuristic:

$$\text{penalty}(u,v) = \begin{cases} 0 & a(v) \text{ is } \textbf{none} \text{ or OPEN}, \\ \texttt{route\_direction\_penalty} & a(v) \text{ is DRAINING}, \\ 0 & \text{the traversal direction matches } a(v)\text{'s}, \\ \texttt{route\_direction\_penalty} & \text{otherwise}, \end{cases} \tag{43}$$

$$\text{cost}(u,v) = 1 + \text{penalty}(u,v), \tag{44}$$

with `route_direction_penalty` $= 6.0$ by default.

The rationale is stated in the module's own docstring: PIBT looks exactly one step ahead, so directional information has to be baked into the *route* itself, or a robot will sit at an aisle entrance repeatedly proposing the same badly-scoring move until the aisle flips. Note that this is a routing preference, not a legality rule: an expensive edge is still an edge, in keeping with Section 3.1.

13 Task assignment

> *Which robot gets which job. A greedy match over a cost that knows about distance, congestion, one-way aisles and how long the job has queued.*

13.1 Directional delay

$$T_{\text{dir}}(s,g) = \sum_{a \in \mathcal{A}(\text{route}(s,g))} \text{delay}(a), \tag{45}$$

$$\text{delay}(a) = \begin{cases} 0 & a \text{ is OPEN}, \\ \text{occ}(a) + 1 & a \text{ is DRAINING}, \\ \frac{1}{2} T_{\min} + \text{occ}(a) & \text{direction mismatches}, \\ 0 & \text{direction already matches}. \end{cases} \tag{46}$$

The third case is the cost of waiting for a drain and a flip. Computed only when `direction_control` is `"aisle"`, with each aisle counted once even if the route touches it repeatedly.

13.2 Blocking estimate

$$B(i,\tau) = \frac{|\{\text{bottleneck vertices on route}(x_i, p_\tau)\}|}{\max(1, |\text{route}|)}. \tag{47}$$

13.3 The assignment cost

$$J(i,\tau) = a\, d(x_i,p) + b\, d(p,d) + g\, C + \delta\, W + e\, T_{\text{dir}} + z\, B. \tag{48}$$

Term	Meaning	Coefficient
$a\,d(x_i, p)$	robot to pickup distance	`assign_alpha_to_pickup` $= 1.0$
$b\,d(p, d)$	pickup to delivery distance	`assign_beta_pickup_to_delivery` $= 0.5$
$g\,C$	route congestion, if congestion-aware	`assign_gamma_congestion` $= 12.0$
$\delta\,W$	task waiting time, capped at 60	`assign_delta_waiting` $= -0.5$
$e\,T_{\text{dir}}$	directional delay, Section 13.1	`assign_eta_direction` $= 1.0$
$z\,B$	bottleneck blocking, if congestion-aware	`assign_zeta_blocking` $= 1.0$

Note that $\delta = -0.5$ is **negative**: a longer-waiting task *lowers* its own cost and is preferred over a fresher, possibly nearer one. This is the assignment layer's own fairness mechanism, separate from and additive to the priority function's waiting term (Section 7). The cost returns ∞ if either leg is unreachable, so an infeasible pairing is never chosen.

> Two coefficients here were recalibrated rather than redesigned, and both mattered. `assign_gamma_congestion` was 2.0 against distances of 10–40, so the entire congestion term was worth about four cost units and could not influence a match — a congestion-aware assignment that cannot change an assignment is not a mechanism. And `assign_waiting_cap` bounds a term that otherwise grows without limit in a lifelong run: uncapped, waiting dominates distance and congestion within about a hundred steps and the match degenerates to oldest-task-first, which is exactly what hypothesis H4 was reporting before the cap existed.

Worked example. Same run, timestep 18. One unassigned task: pickup (0,0), delivery (15,0), released at t = 6 so it has been waiting 12 steps. Two candidate robots, costed by `TaskAssigner.assignment_cost`:

```
robot           a·d(r,p)  b·d(p,d)  g·C    d·W  e·T_dir  z·B  J(i,τ)
--------------  --------  --------  -----  ---  -------  ---  ------
robot 16 (0,4)  4         7.5       2.4    −6   0        0    7.9
robot 18 (0,8)  8         7.5       2.667  −6   0        0    12.17
```

Robot 16 wins at J = 7.9. Two columns are worth pausing on. `d(pickup, delivery)` is identical for both robots, because it does not depend on which robot goes — it is in the cost only so that a queue of tasks is ordered by total work rather than by how near its pickup happens to be. And $\delta = -0.5$ is *negative*, so the waiting term is a discount: the longer a task has been queued the cheaper it looks, which is how ageing gets into a greedy match.
That discount is capped at `assign_waiting_cap` = 60. Uncapped, `waiting_time` grows without bound in a lifelong run and dwarfs both distance and congestion within about a hundred steps; the match degenerates into oldest-task-first, and a congestion-aware assignment can no longer show any effect at all — which is exactly what H4 reported before the cap existed (Section 13.3).

13.4 Greedy matching

`assign_tasks_greedily` first pre-filters the unassigned pool to the oldest

$$\max(\texttt{assignment\_candidate\_limit},\ |\text{free robots}|)$$

tasks, sorted by $(t_{\text{rel}}, \text{id})$ — a cheap bound, so cost is never computed against an unbounded backlog. It then repeatedly takes the cheapest feasible pair:

Algorithm 5 `assign_tasks_greedily`

while free robots and candidate tasks remain **do**
  $(i^*, \tau^*) \leftarrow \arg\min_{(i,\tau)\ \text{feasible}} J(i, \tau, t)$
  assign $i^* \leftarrow \tau^*$; remove both from their pools
end while

This is greedy minimum-cost matching, $O(RT)$ per round for R robots and T candidate tasks — not the Hungarian algorithm, so it is locally greedy at each pairing rather than globally optimal. It is held *identical* across every planner compared in Section 21, including both external baselines, so that comparisons isolate the movement layer.

14 Deadlock detection and recovery

> *How the system tells a genuine deadlock apart from ordinary queueing — which in dense lifelong traffic is most of what it sees — and the seven increasingly drastic things it tries once it is sure.*

14.1 Three distinct stall signals

Three related but different counters live on `Robot`, and they are worth disambiguating precisely because they feed different formulas:

Counter	Meaning	Reset
`waiting_time`	steps spent physically stationary while not yet at the waypoint	on any move, or on reaching the waypoint
`blocked_time`	consecutive steps the PIBT recursion returned INVALID	on any VALID result
`no_progress_steps`	consecutive steps $D_w(x_i)$ did not strictly decrease	on any route-distance improvement

14.2 Progress signal

$$\Delta_i(t) = D_w(x_i(t-1)) - D_w(x_i(t)). \tag{49}$$

`update_progress` treats a robot as not stalled whenever it is idle, already at its waypoint, or at distance 0; otherwise it increments `no_progress_steps` unless the distance strictly decreased and both ends were finite. Then

$$\text{is\_blocked}(i) \iff \texttt{no\_progress\_steps} \geq t_{\text{blocked}} \ \vee \ \texttt{blocked\_time} \geq t_{\text{blocked}}, \qquad t_{\text{blocked}} = 10. \tag{50}$$

14.3 Repeated configurations and the wait-for graph

Repeated configuration. With period $k = 4$, on the 16-entry position history deque:

$$\text{repeated}(i) \iff \text{history}[-k{:}] = \text{history}[-2k : -k], \tag{51}$$

that is, the robot's last k positions exactly equal the k before them.

Wait-for dependency graph. An edge $i \to j$ exists exactly when robot i's `waiting_for_robot`, set during the priority-inheritance step of Algorithm 4, is robot j. `find_cycles` runs an iterative white/grey/black DFS to find every simple directed cycle. A cycle here is a genuine circular wait — i waits for j, …, waits for i — the textbook signature of deadlock.

`detect_deadlocked_groups` combines two signals: any cycle in the wait-for graph containing at least one currently-blocked robot, and a BFS-connected-component clustering of the *remaining* blocked robots within $\max(2, R_{\text{local}})$ hops of each other. The second pass catches deadlocks that do not present as a clean wait-for cycle — several robots each stuck against static geometry in the same area — without needing a global traversal. Group lifetimes are

tracked by `frozenset` membership keys across timesteps, which is what makes "detected", "recovered" and "unrecovered" distinguishable.

> **Corroboration.** With `require_deadlock_corroboration` (the default), a group must show the progress signal *and* at least one of a wait-for cycle or a repeated configuration. The specification named three stall signals and the implementation treated the weakest as sufficient — which in dense lifelong traffic describes an ordinary queue, not a deadlock. Recovery therefore escalated on healthy robots, and levels 5–7 below cost far more than they saved: 0.022 tasks per step against 0.134 on `warehouse_corridors`. Section 3.3 explains why this stayed invisible for so long.

14.4 Seven-level progressive recovery

`recover_from_deadlock` escalates **one level per timestep** a given group persists, applying the next level's remedy each time the group is still detected:

1. **Recompute routes** — from scratch for every robot in the group. The cheapest possible fix: perhaps the route was merely stale.
2. **Recompute affected aisles** — force `lock_until` $= t$ on the group's current and next aisles, allowing an immediate re-decision instead of waiting out the lock; reopen an affected aisle if it is empty, force it to DRAINING if it is occupied and directional.
3. **Release stale reservations** held by the group.
4. **Raise blocked priorities** — $p_i \mathrel{+}= k_b \cdot$ `no_progress_steps`, an escalating boost on top of Equation (18).
5. **Allow temporary reverse movement** — `allow_reverse_until` $= t + t_{\text{blocked}}$, with a shorter grace period for ignoring direction entirely.
6. **Assign escape vertices** — route each member to its nearest unoccupied, unclaimed parking or passing bay, falling back to any degree-≥ 3 vertex, and set its state to RECOVERY, which also grants $P_{\text{emergency}}$.
7. **Run the local fallback planner** — the last resort: drop all directional constraints for the group for t_{deadlock} steps, release reservations, restore task-directed state for anyone mid-recovery.

If a group is still detected after level 7, that remedy is reapplied each subsequent timestep and the group is counted once, not repeatedly, as unrecovered.

Why one level per timestep and not all seven per call. Progress from a remedy can only be *observed* on the next timestep, because the simulator is synchronous. Running all seven in one call would apply six of them blind. "Progressive" recovery in a synchronous loop necessarily means one escalation per detection cycle.

15 The lifelong simulation loop

> *The sixteen steps that run, in this order, on every timestep. If you read one section to understand the system's shape, read this one.*

`Simulator.step()` executes the following, tying every prior section together in execution order.

Algorithm 6 `Simulator.step()` — one timestep

1: rebuild the occupancy index; clear the per-timestep congestion cache ▷ Section 9.1
2: receive newly arrived tasks ▷ Section 6.3
3: update task and robot state transitions; record completions; release aisle reservations for robots that just delivered
4: assign tasks greedily; park idle robots ▷ Section 13
5: update waypoints, route distances, proximity mode, β_i, γ_i, current aisle ▷ Section 8.1
6: compute routes; find each robot's next aisle; compute the preferred direction with hysteresis ▷ Sections 8.7 and 12
7: update aisle queues, direction decisions and reservation grants ▷ Sections 10.3 and 10.5
8: detect deadlocks and recover, using the *previous* step's wait-for graph ▷ Section 14
9: compute priorities, sort descending, grant reservations in that order ▷ Section 7
10: run the low-level planner: PIBT, or a baseline via `planner_factory` ▷ Sections 11 and 18
11: validate the joint plan, raising on any residual vertex or swap conflict ▷ structurally impossible; checked anyway
12: execute every move atomically in one synchronised pass
13: update waiting time, current aisle, route distance and the progress signal; release reservations for robots now outside every aisle
14: rebuild the occupancy index ▷ positions changed
15: log the timestep's metrics
16: append a snapshot to the history, if recording

Step 8 deliberately reads the *previous* step's wait-for graph, because `waiting_for_robot` is only set during that step's PIBT run (Algorithm 4); there is no earlier point at which the edges exist.

Idle robots. `_park_idle_robots` sends task-less robots to the nearest unclaimed parking or passing bay. If none is free, the robot *holds its current position* rather than drifting toward an intersection: occupying an intersection throttles every route through it, and priority inheritance will displace an idle robot the moment a busier one needs that cell anyway. This is a small decision with a large effect on a congested floor.

Recursion headroom. `_raise_recursion_limit` sets Python's recursion limit to $\max(\text{current}, 100 + 12n)$ for n robots. PIBT's inheritance chains can, in the worst case, be as deep as the fleet.

`run(max_timesteps, until_tasks, progress)` loops to the horizon, stopping early if a completion target is reached or — in non-lifelong mode only — every robot has reached its static goal.

16 Evaluation metrics

> *What gets measured at the end of a run, how each number is defined, and which of them may be compared with which.*

16.1 Percentile

Standard linear interpolation, on the sorted values, with $\text{pos} = (n-1) \cdot q/100$ and $\text{lo} = \lfloor \text{pos} \rfloor$:

$$\text{percentile}(x, q) = x_{\text{lo}}(1 - f) + x_{\text{lo}+1} f, \qquad f = \text{pos} - \text{lo}. \tag{52}$$

16.2 Jain's fairness index

$$\mathcal{J} = \frac{\left(\sum_i T_i\right)^2}{n \sum_i T_i^2}, \tag{53}$$

computed on each robot's completed-task count (Jain et al., 1984). $\mathcal{J} \in (0, 1]$, equal to 1 exactly when every robot completed the same number of tasks, and tending to $1/n$ as one robot does everything.

16.3 Aggregate report fields

$\Theta = K/\max(1, T)$ (Equation (5)); makespan as the maximum completion time; mean, median, p_{95} and maximum service time over completed tasks; total and mean travel distance, incremented once per actual move; direction switches per 1000 steps; the PIBT counters of Section 11.4; and the deadlock counters of Section 14.3, with mean recovery time as recovery time total/recovered.

Four counters exist specifically to test claims that throughput cannot see, and are what the hypothesis suite of Section 21.6 scores against:

Counter	The claim it tests
`head_on_conflicts`	entry admission reduces head-on meetings (H3)
`counterflow_moves`	the measured price of ranking direction rather than enforcing it
`aisle_throughput_per_1000`	narrow aisles flow better under direction control (H1)
`starvation_flips`	flips forced by the maximum green rather than by demand

A companion note, `docs/metrics.md`, catalogues every one of these quantities with the way it misleads if read alone, and reports the three the verdict actually turns on — travel distance per delivered task, head-on conflicts per 1000 steps, and throughput against the published baselines — with their measured values. It is generated from the same dataset as Section 21.

> **Service time is censored.** Every service-time field is computed over *completed* tasks only, and in the saturated regime of Section 1.4 that is a minority of the tasks released. A planner that completes only the easy ones reports the best mean service time in the table while delivering nothing. Compare service times only between planners of comparable throughput; Figure 2 shows what happens otherwise.

17 Statistics

> *How to tell a real difference between two configurations from noise, without assuming the measurements are normally distributed.*

Pure Python, no SciPy or NumPy, so the analysis carries the same zero-dependency guarantee as the simulator. Used to turn a handful of per-seed means into a defensible interval and significance test.

17.1 Percentile bootstrap confidence interval

Algorithm 7 `bootstrap_ci(values,` $B = 10{,}000$`,` $\alpha = 0.05$`)`

for $b = 1$ to B **do**
    draw n values from `values` *with replacement*; record their mean
end for
sort the B resample means
return $\left(\overline{x},\ m_{\lfloor(\alpha/2)B\rfloor},\ m_{\lfloor(1-\alpha/2)B\rfloor}\right)$

This is the standard percentile bootstrap for a $(1-\alpha)$ interval on the mean. No distributional assumption is made beyond resampling with replacement approximating the sampling distribution. It degenerates to a point for fewer than two values, since no variance can be estimated from one sample.

17.2 Permutation test

Algorithm 8 `permutation_test(a, b,` $P = 10{,}000$`)`

obs $\leftarrow \overline{a} - \overline{b}$; pooled $\leftarrow a \cup b$; $N \leftarrow \binom{|a|+|b|}{|a|}$
if $N \leq 20{,}000$ **then**
    $p \leftarrow \dfrac{|\{\text{splits with } |\text{mean difference}| \geq |\text{obs}| - 10^{-12}\}|}{N}$ ▷ exact enumeration
else
    $p \leftarrow \dfrac{|\{\text{of } P \text{ random relabelings meeting the same bound}\}|}{P}$
end if
return (obs, p)

A standard two-sided permutation test: under the null hypothesis that a and b are drawn from the same distribution, every relabeling of the pooled data into two groups of the original sizes is equally likely, so p is the fraction of relabelings at least as extreme as the one observed. Below the enumeration threshold every split is enumerated exactly and the result is seed-independent; above it, relabelings are sampled from a seeded generator, so the result is still deterministic. It returns $p = 1$ if either group is empty or spans the whole pool.

> With five seeds per cell, $\binom{10}{5} = 252$ distinct splits exist, so the smallest attainable two-sided p-value is $2/252 \approx 0.008$. A reported $p < 0.001$ in Section 21 therefore comes from a cell with more seeds or from the Monte Carlo branch; a five-seed comparison cannot produce one, and none is claimed.

18 Baseline algorithms

> *The two published lifelong algorithms TOLL-PIBT is compared against, and the space-time search both are built on. Section 2 says what they are for; this section says how they are implemented here.*

Both reuse the identical `assignment.TaskAssigner` (Section 13), so only the low-level movement and collision-avoidance layer differs. Neither uses candidate scoring, aisle direction or reservations; both navigate with their own time-expanded, reservation-table A*, built from scratch because nothing else in this codebase is collision-aware in that sense.

18.1 Time-expanded A*

The shared primitive: cooperative A* over states (vertex, t) rather than plain vertices, since avoiding other robots requires reasoning about *when* a cell is occupied, not just whether it is passable.

$$f(v,t) = g(v,t) + h(v), \qquad h(v) = D_{\text{goal}}(v), \tag{54}$$

with h the precomputed BFS distance map of Section 5.2. On an unweighted grid where every move costs exactly 1, h is the true shortest-path distance and therefore admissible and consistent, so the search is optimal for the single agent. Each transition is a move to a neighbour or a wait, each costing +1, filtered through the reservation table — which checks both a vertex hold and, critically, a reverse-edge hold, catching swap conflicts the same way Section 11.3 does.

Two modes:

- `require_goal=True` (Token Passing): return only a path that reaches the goal *and* can hold there for the rest of the horizon, the standard cooperative-A* goal condition, so a robot does not stop somewhere another robot will need later.
- `require_goal=False` (RHCR): the horizon is a short rolling window decoupled from goal distance, so reaching the goal within one window is the exception. Returns the best-effort path to whichever boundary state has the smallest h, ties broken by pop order.

`prioritized_plan` plans robots one at a time, committing each path into the shared table before planning the next — the classical sequential MAPF decomposition: no joint-optimality guarantee, always fast, always produces some legal plan. Every robot's current cell is pre-held for one extra timestep so a later, not-yet-planned robot cannot path straight through a robot that has not moved yet.

`resolve_residual_conflicts` is a post-hoc safety net run after either baseline finishes: it repeatedly finds any residual vertex or swap conflict among the committed next positions and downgrades the lower-priority robot to hold in place, iterating until stable — bounded by $n + 1$ iterations, since each pass removes at least one conflict.

18.2 Token Passing

Each step, any robot whose committed path is stale (start or goal mismatch, or exhausted) is replanned via `prioritized_plan` with `require_goal=True`, against a table pre-seeded by holding every still-valid path of settled robots. A robot whose search fails waits in place, incrementing `tp_path_not_found`.

Token Passing as published has **no priority-inheritance or backtracking mechanism**: a blocked robot cannot ask the robot in its way to move. This is the structural reason — not an artefact of this implementation — for the corridor gridlock reported in Section 21.2: once a queue forms in a single-file corridor with no mechanism to displace anyone, it never resolves. The counters `tp_path_not_found` and `tp_forced_holds` are exposed so that a zero-throughput run can be confirmed as genuine gridlock rather than a crash or a silent no-op.

18.3 RHCR

Every `baseline_replan_period` steps (default 5) all robots are jointly replanned over a `baseline_window`-length horizon (default 10) via windowed CBS:

Algorithm 9 Windowed CBS (`baselines/rhcr.py`)

root: each robot's unconstrained time-expanded A* path (`require_goal=False`); cost = $\sum_i |\text{path}_i|$
while the open list is non-empty **do**
    pop the lowest-cost node
    $c \leftarrow$ first conflict in the node's paths within the window ▷ shared vertex, or a reversed-edge swap
    if no conflict **then return** the node's paths
    end if
    branch into two children, each adding one of the two constraints from c
    replan only the newly-constrained robot in each child; push if feasible
end while

Branching follows Sharon et al. (2015) exactly: a vertex conflict (i, j, v, t) gives children $\{i$ forbidden at $(v, t)\}$ and $\{j$ forbidden at $(v, t)\}$; an edge conflict gives the two directed-edge prohibitions. If node expansions exceed `node_expansion_cap` (500) before a conflict-free node is found, the search falls back to `prioritized_plan` for that window and increments `cbs_fallbacks` — standard, documented practice for bounding CBS's worst case, not a shortcut unique to this implementation.

Between periodic replans, `_repair_stale_paths` gives any robot with a newly assigned task or an exhausted path an immediate single-agent repair, rather than making it wait up to a full replan period.

19 Parameter reference

> *Every tunable in one place: what it does, what it defaults to, and which ablation variant switches it.*

19.1 Defaults by section

Parameter	Default	Section
Candidate scoring (Section 8)		
`alpha_progress`	10.0	progress weight α
`beta_strong`	3.0	direction weight ceiling $\beta_{\max}$
`beta_weak`	1.0	defined but read by no formula
`gamma_strong` / `gamma_weak`	2.0 / 0.5	aisle continuity
`lambda_turn` / `lambda_reverse`	0.5 / 2.0	turning cost
`mu_congestion`	1.0	congestion weight μ
`nu_wait`	0.2	wait penalty
`xi_bottleneck`	1.0	bottleneck penalty
`zeta_counterflow`	8.0	counterflow price (Section 3.1.3)
`zeta_reservation`	8.0	unticketed-entry price
`progress_normalization`	`"step"`	Section 8.5
`hard_direction_constraints`	`False`	restores rejection, for comparison
Congestion (Section 9)		
`omega_local/aisle/downstream`	1.0 each	mixture weights
`local_congestion_radius`	3	R_{local}
`downstream_horizon`	5	lookahead K

Parameter	Default	Section
`congestion_normalisation`	`True`	Section 9.3
Proximity (Section 8.1)		
`r_near` / `r_far`	2 / 8	mode thresholds
Aisle management (Section 10)		
`minimum_aisle_lock_time`	20	$T_{\min}$
`maximum_aisle_lock_time`	40	$T_{\max}$, the maximum green
`direction_switch_threshold`	5.0	dead band τ_{switch}
`aisle_capacity`	10	cap on any aisle's capacity
`aisle_capacity_model`	`"drain"`	Section 5.4
`aisle_capacity_ratio`	1.0	capacity per unit length
`reservation_ttl`	15	ticket lifetime
`max_drain_time`	30	force-reopen a stuck drain
`directional_aisle_min_length`	4	below this, always OPEN
`direction_aware_routing`	`False`	Section 12
`route_direction_penalty`	6.0	A* edge penalty
`parity_bias`	0.0	off: does not fix parallel-corridor lock-step
`coordinate_aisle_directions`	`False`	off: a null result (Section 10.4.1)
`demand_spread`	`False`	off: net negative (Section 10.3)
Directional demand (Section 10.2)		
`w_urgency` / `w_waiting`	1.0 / 0.5	
`w_proximity`	2.0	
`w_route_length` / `w_congestion`	0.05 / 0.5	
Priority (Section 7)		
`priority_emergency`	400.0	
`priority_loaded` / `priority_pickup`	300.0 / 200.0	
`priority_repositioning` / `priority_free`	100.0 / 0.0	
`priority_inside_aisle`	50.0	
`waiting_weight`	5.0	sets the 80-step fairness horizon
`blocked_weight` / `urgency_weight`	10.0 / 1.0	
Deadlock (Section 14)		
`t_blocked` / `t_deadlock`	10 / 20	
`require_deadlock_corroboration`	`True`	Section 3.3
`config_history_length`	8	reserved; read by no formula
Task assignment (Section 13)		
`assign_alpha_to_pickup`	1.0	
`assign_beta_pickup_to_delivery`	0.5	
`assign_gamma_congestion`	12.0	was 2.0, too small to matter
`assign_delta_waiting`	−0.5	negative: ageing discount
`assign_waiting_cap`	60.0	bounds an unbounded term
`assign_eta_direction` / `assign_zeta_blocking`	1.0 / 1.0	
`assignment_candidate_limit`	32	
Ablation switches		
`lifelong`	`True`	

Parameter	Default	Section
`direction_control`	`"aisle"`	`none` | `robot` | `aisle`
`hysteresis`	`True`	
`congestion_scoring` / `congestion_assignment`	`True`	split from one flag
`reservations` / `recovery` / `turning_cost`	`True`	
Simulation and baselines		
`max_timesteps` / `seed`	500 / 0	
`park_when_idle` / `validate_every_step`	`True`	
`baseline_window` / `baseline_replan_period`	10 / 5	RHCR only

Table 3: Every field of `config.Params`, with its default.

`congestion_aware` survives as a read/write alias setting both congestion flags, so saved configurations and CLI overrides keep working. It was one flag switching three mechanisms — the movement penalty, the assignment congestion term and the assignment blocking term — which made hypothesis H4, a claim about matching alone, unmeasurable.

19.2 The ablation ladder and the factorial designs

`config.ABLATIONS` defines six cumulative presets, each enabling one more mechanism than the last:

`pibt_baseline` → `lifelong_pibt` → `directional_pibt` → `hysteresis_pibt` → `aisle_managed_pibt` → `full_lda_pibt`

Three of those five steps flip *two* flags at once, so a throughput delta on them cannot be attributed to either flag. `FACTORIAL_DESIGNS` resolves each with a 2×2 decomposition:

$$\begin{aligned} \text{main}_A &= \text{value}(A) - \text{value}(\text{base}), \\ \text{main}_B &= \text{value}(B) - \text{value}(\text{base}), \\ \text{additive} &= \text{value}(\text{base}) + \text{main}_A + \text{main}_B, \\ \text{interaction} &= \text{value}(AB) - \text{value}(\text{base}) - \text{main}_A - \text{main}_B. \end{aligned} \tag{55}$$

A large interaction means the flags are strongly non-additive, and the ladder's combined rung cannot be read as one contribution plus the other.

> Two cautions, both learned the hard way and both worth carrying to any factorial:
> **A factorial is only readable if all four corners are distinct runs.** The `aisle_direction_vs_reservations` design was unreadable for exactly one reason: `reservations_only` sets `direction_control` to `"robot"`, and the reservation layer was gated on `direction_control` = `"aisle"`, so factor B was a bit-identical copy of the base and its main effect was *necessarily* zero. "Exactly zero effect" was a property of the wiring. Before reading any decomposition, check that the isolated cell actually differs from the base.
> **A flag that switches more than one mechanism cannot be a factor.** `congestion_aware` moved the movement score, the assignment cost and the assignment blocking term together — the same confound a factorial exists to remove, one level down.

`config.PAIRED_DESIGNS` covers the cases where a 2×2 is impossible because the second factor is undefined when the first is off: there is no "hard constraint" cell without aisle direction,

and no "corroboration" cell without recovery. Those are reported as single-factor comparisons with the same bootstrap interval and permutation test.

20 Glossary of symbols

> *Every symbol used above, in one table, with the module that owns it.*

Symbol	Meaning	Module
$x_i(t)$	position of robot i at time t	`robot.py`
$D_g(v)$	BFS graph distance from v to goal g	`graph.py`
$\Delta(G)$	maximum vertex degree in the graph	`graph.py`
$N(v)$	the 4-connected neighbours of v	`graph.py`
C_i	PIBT candidate set, $N(x_i) \cup \{x_i\}$	`pibt.py`
r_i	route distance from robot i to its waypoint	`scoring.py`
$R_{\text{near}}, R_{\text{far}}$	proximity-mode thresholds (2, 8)	`config.py`
β_i	smooth direction-preference weight	`scoring.py`
γ_i	aisle-continuity weight	`scoring.py`
$\alpha, \lambda_{\text{turn}}, \mu, \nu, \xi$	score weights: progress, turn, congestion, wait, bottleneck	`scoring.py`
ζ	counterflow and unticketed-entry price	`scoring.py`
λ_{reverse}	180°-reversal turn-cost multiplier	`scoring.py`
$S_i(v)$	full candidate score for robot i moving to v	`scoring.py`
$C_i(v)$	congestion estimate at candidate v	`congestion.py`
$\omega_{\text{loc}}, \omega_{\text{aisle}}, \omega_{\text{down}}$	congestion mixture weights	`config.py`
S_a^+, S_a^-	aggregate forward / reverse demand for aisle a	`aisle_manager.py`
$\rho_i(a)$	how much of robot i's demand is charged to aisle a	`aisle_manager.py`
τ_{switch}	direction-switch dead band	`aisle_manager.py`
$T_{\text{min}}, T_{\text{max}}$	minimum lock and maximum green	`config.py`
$p_i(t)$	robot i's scheduling priority at time t	`priority.py`
P_{class}	task-class base priority	`priority.py`
k_w, k_b, k_u, k_e	priority weights on waiting, blocked, urgency, in-aisle	`priority.py`
ε_i	per-robot tie-breaker, $10^{-4} \cdot \text{id}$	`robot.py`
U_i	task urgency	`task.py`
W_i	waiting term (robot waiting time in priority; a stall-aware maximum in the aisle layer)	`priority.py`, `aisle_manager.py`
$J(i, \tau)$	task-assignment cost	`assignment.py`
T_{dir}	expected aisle-direction delay along a route	`assignment.py`
$B(i, \tau)$	route bottleneck-blocking estimate	`assignment.py`
$\Delta_i(t)$	one-step route-distance progress	`deadlock.py`
$t_{\text{blocked}}, t_{\text{deadlock}}$	stall and deadlock thresholds	`config.py`
Θ	throughput, tasks per timestep	`metrics.py`
$\mathcal{J}$	Jain's fairness index over completion counts, $\in (0, 1]$	`metrics.py`
λ	task arrival rate, tasks per timestep	`task.py`
∞	the unreachable-distance sentinel	`types.py`

Table 4: Every symbol, and the module that owns it.

Part III
What the measurements said

21 Results

> *Five suites, five seeds, 400 timesteps, four maps. The aisle layer is ahead on the two aisle-shaped maps and behind on the two open ones; both external baselines are far behind everywhere; and the mechanisms Section 3 argues for are the largest effects in the system by a wide margin.*

21.1 Setup

Everything below comes from one dataset, `docs/data/`, written by `experiments/run_all.py` and committed alongside this document, so every figure regenerates from the same runs.

Map	Shape	Robots	λ	Served fraction
`warehouse_bottleneck`	two halves, one joining corridor	16	0.8	18%
`warehouse_corridors`	five parallel single-file corridors	35	1.0	13%
`warehouse_narrow`	four 5-cell aisles per bank	30	1.2	—
`warehouse_medium`	open grid	40	1.5	33%

Table 5: The scenarios. "Served fraction" is the best planner's completed tasks over the tasks the arrival process released — the saturation of Section 1.4, measured.

Five seeds per cell, Poisson arrivals, and the statistics of Section 17 throughout. One consequence of five seeds is worth stating before any number is read: with five values a side the exact permutation test has $\binom{10}{5} = 252$ label splits, so **the smallest attainable two-sided p-value is** $2/252 \approx 0.008$, and a genuine difference of moderate size can easily land at $p \approx 0.06$. That is exactly what happens to this method's two wins, and Section 21.8 says so rather than rounding it away.

21.2 Against the published baselines

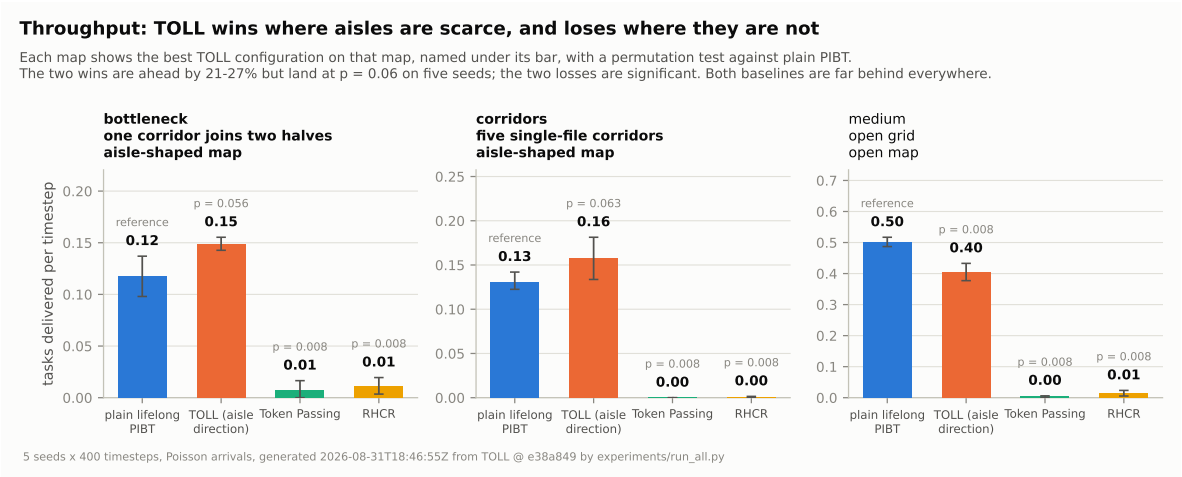

Figure 1: Throughput on the three maps the baseline suite covers.

Map	Planner	Θ	svc	ms/step	p vs PIBT
bottleneck	plain lifelong PIBT	0.118	154.2	0.40	—
	TOLL (full)	**0.145**	154.3	1.45	0.095
	Token Passing	0.007	31.1	36.2	0.008
	Token Passing + recovery	0.008	42.2	32.3	0.008
	RHCR	0.011	22.6	75.0	0.008
corridors	plain lifelong PIBT	0.131	157.0	0.84	—
	TOLL (full)	0.121	148.9	2.84	0.492
	Token Passing	0.000	—	80.8	0.008
	Token Passing + recovery	0.000	—	40.3	0.008
	RHCR	0.001	8.6	43.0	0.008
medium	plain lifelong PIBT	**0.502**	139.0	1.03	—
	TOLL (full)	0.313	150.0	4.36	0.008
	Token Passing	0.004	49.9	222.2	0.008
	Token Passing + recovery	0.027	144.9	191.2	0.008
	RHCR	0.014	38.7	97.9	0.008

Table 6: Five seeds, 400 timesteps. Θ is tasks per timestep; "svc" is mean service time over completed tasks only (Section 16); p is a two-sided permutation test against plain lifelong PIBT on throughput. A dash in the service-time column means the planner finished no tasks at all, so the statistic does not exist.

Three things in that table are worth reading carefully.

Both baselines lose by two orders of magnitude, significantly, everywhere. Every baseline cell sits at $p = 0.008$, the floor of a five-seed test. Token Passing completes *literally zero* tasks on `warehouse_corridors` across all five seeds. Its counters (`tp_path_not_found`, `tp_forced_holds`) confirm genuine gridlock rather than a crash: robots queue nose-to-tail and no robot can reserve a path through the robots ahead of it, for the rest of the run. This is the structural failure of Section 2.3, and the first animation in `docs/gifs/` shows it beside the same scenario under priority inheritance.

The service-time column is a trap, and this table is where it springs. RHCR reports a mean service time of 8.6 on `warehouse_corridors` — easily the best figure in the table — while delivering 0.001 tasks per timestep. It is not fast; it is selective, and only by accident. Token Passing's dashes are the same phenomenon taken to its limit.

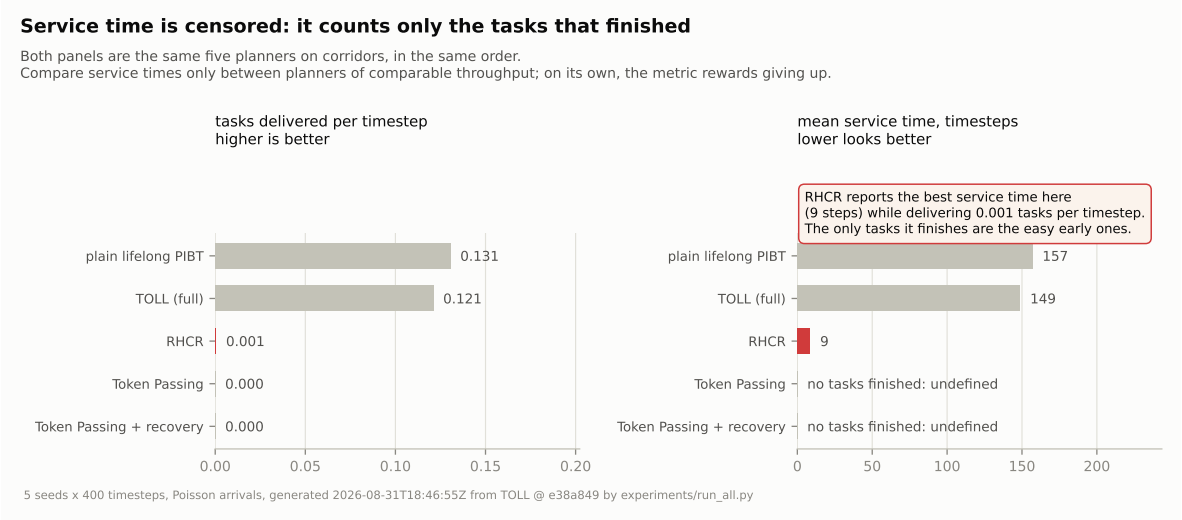

Figure 2: The same five planners, twice. Ranking by the right panel alone inverts the ranking that matters.

Cost does not buy throughput here. Figure 3 puts the two together. The aisle layer costs 1.4–4.4 ms per timestep against plain PIBT's 0.4–1.0; both baselines cost 32–222 ms and deliver almost nothing at this scale. Section 21.10 states what that does and does not say about RHCR.

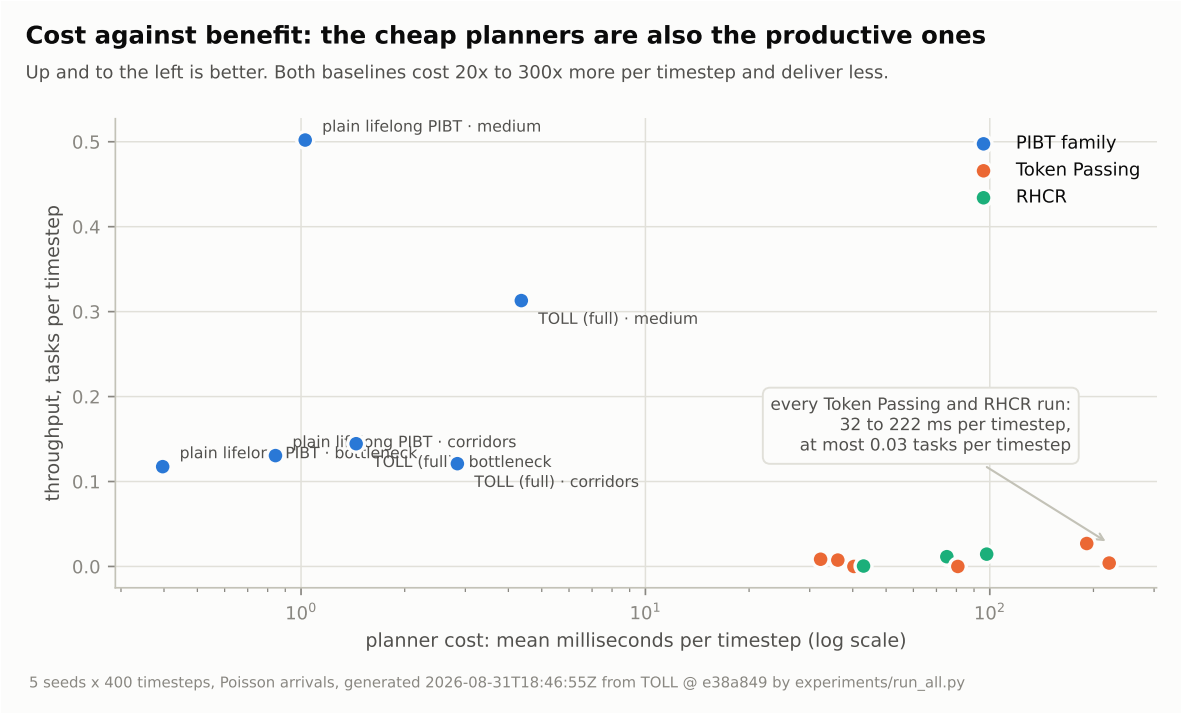

Figure 3: Planner cost against what it delivers. Note the log scale.

21.3 Saturation, in one picture

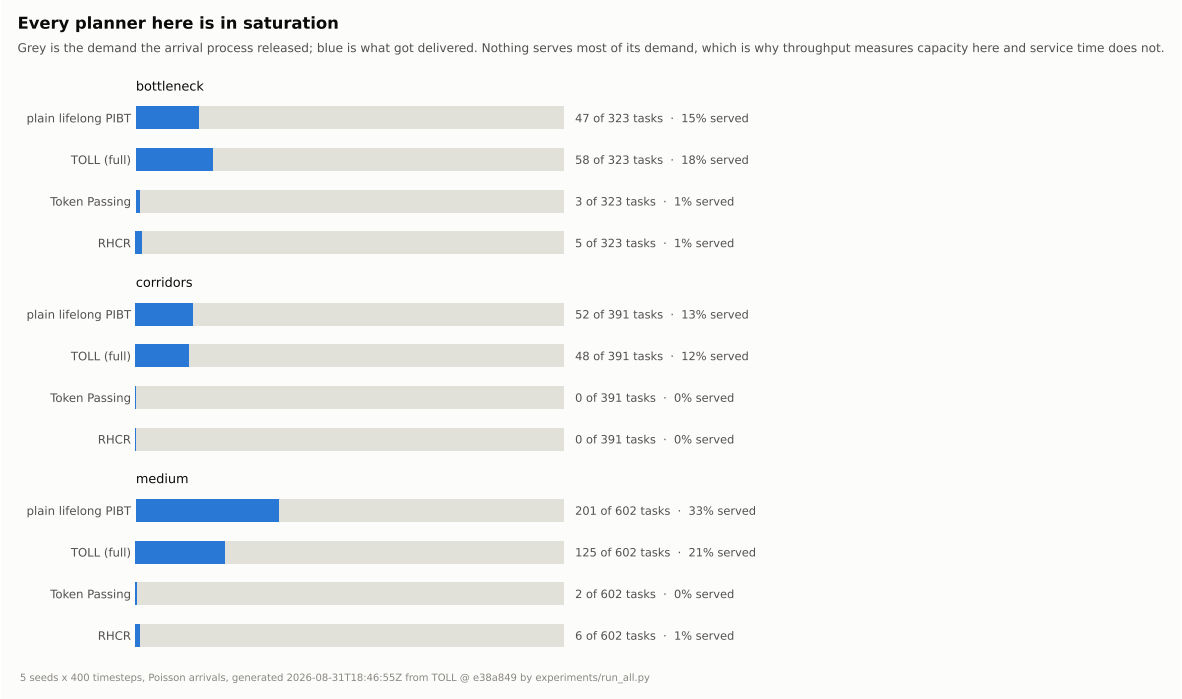

Figure 4: Offered demand against served demand. Between 13% and 33% of the tasks released are delivered; the rest are still queued when the horizon ends.

This is Section 1.4 measured rather than asserted, and it is the context every other number needs. Throughput here is capacity, not activity.

21.4 Which mechanisms earn their cost

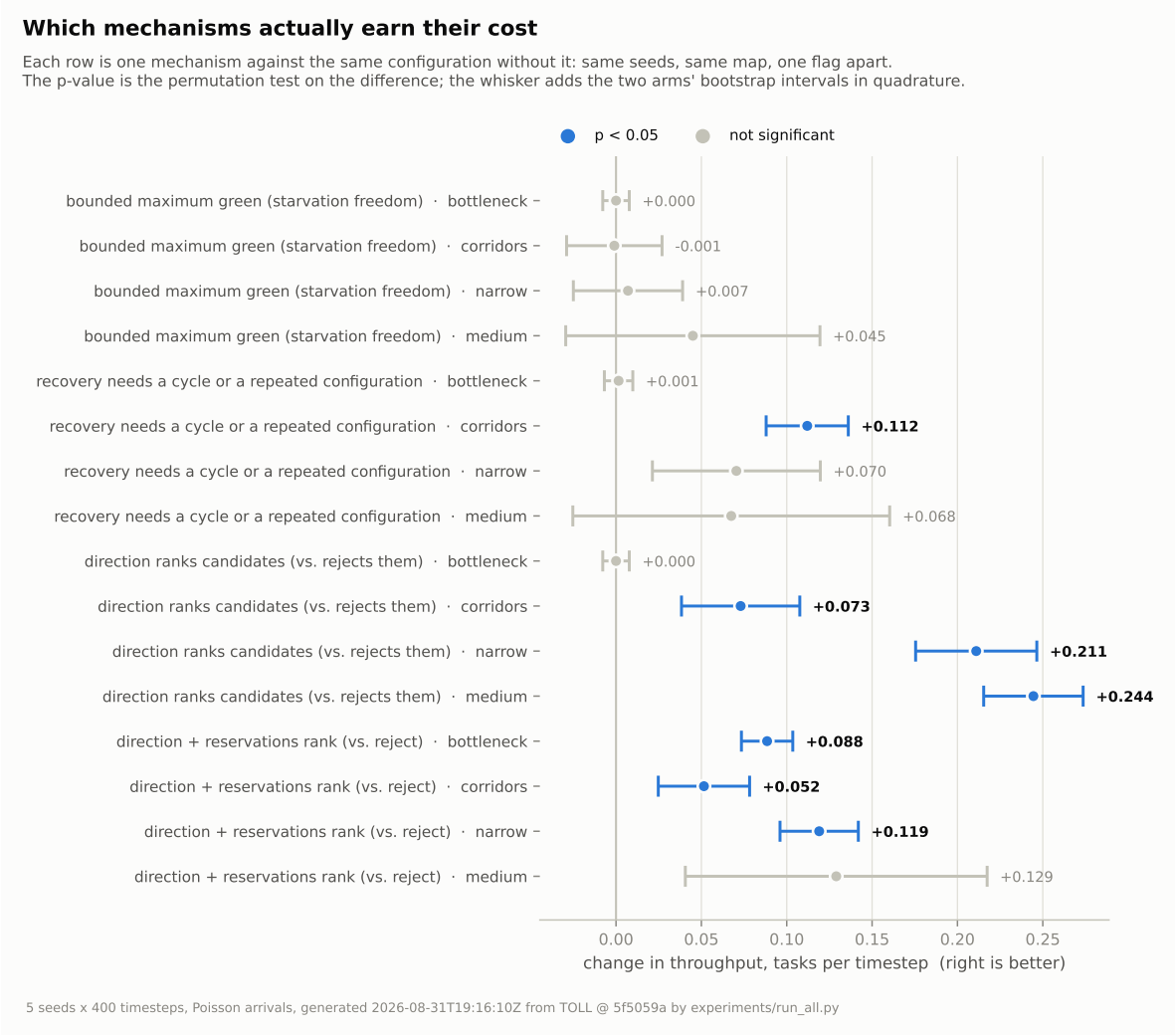

Figure 5: Each of the four repaired mechanisms against the same configuration without it, on every map. Blue is $p < 0.05$.

Mechanism (treatment vs. control)	bottleneck	corridors	narrow	medium
direction **ranks** vs **rejects**	0.149 / 0.149	**0.158 / 0.084**	**0.289 / 0.078**	**0.405 / 0.161**
same, with reservations	**0.149 / 0.060**	**0.095 / 0.043**	**0.152 / 0.033**	0.217 / 0.088
bounded maximum green	0.149 / 0.149	0.158 / 0.159	0.289 / 0.282	0.405 / 0.360
recovery needs corroboration	0.149 / 0.148	**0.134 / 0.022**	0.204 / 0.134	0.245 / 0.177

Table 7: Throughput, treatment / control, five seeds. **Bold** is $p \leq 0.05$. `config.PAIRED_DESIGNS`.

Direction as a price rather than a rule is the largest effect in the system. On the three maps where an aisle ever commits a direction it is worth 1.9× (corridors), 2.5× (medium) and 3.7× (narrow) throughput, each at $p = 0.008$; with entry admission also on, the range runs to 4.6×. On `warehouse_bottleneck` the two arms are identical to three decimals, and that is not a null result: 16 robots spread over 25 aisles never push the demand imbalance past $\tau_{\text{switch}} = 5.0$, so no aisle ever commits a direction and there is nothing for the treatment to change. **That cell is an experiment that did not run**, and is reported as such rather than as evidence of no effect.

This is Section 3.1 measured. The second animation in `docs/gifs/` is the corridors pair, side by side.

Corroborated stall detection is the second. On `warehouse_corridors`, 0.134 against 0.022 tasks per timestep ($p = 0.008$): a six-fold difference obtained entirely by refusing to escalate on the weakest of the three stall signals. On the other maps the effect points the same way and does not clear five-seed significance ($p = 0.056$ on narrow, 0.230 on medium). The mechanism matters most exactly where deadlocks are easiest to manufacture.

The maximum green is not significant on throughput, and that is the finding. Its throughput effect is between -0.001 and $+0.045$, with p from 0.437 to 0.968. It was decisive while direction was a hard constraint, because a starved robot then had no way out at all; once counterflow is merely expensive, a robot can buy its way out of a starved aisle and liveness no longer rests on the signal. The two fixes are partly redundant. What the maximum green does change is the signal itself: on `warehouse_narrow` it produces 22.5 direction switches per 1000 steps against 1.0 without it ($p = 0.016$), of which 5.2 per run are flips forced by the rule rather than by demand (`starvation_flips`). Without it, those aisles simply never turn.

21.5 The ablation ladder is not monotone

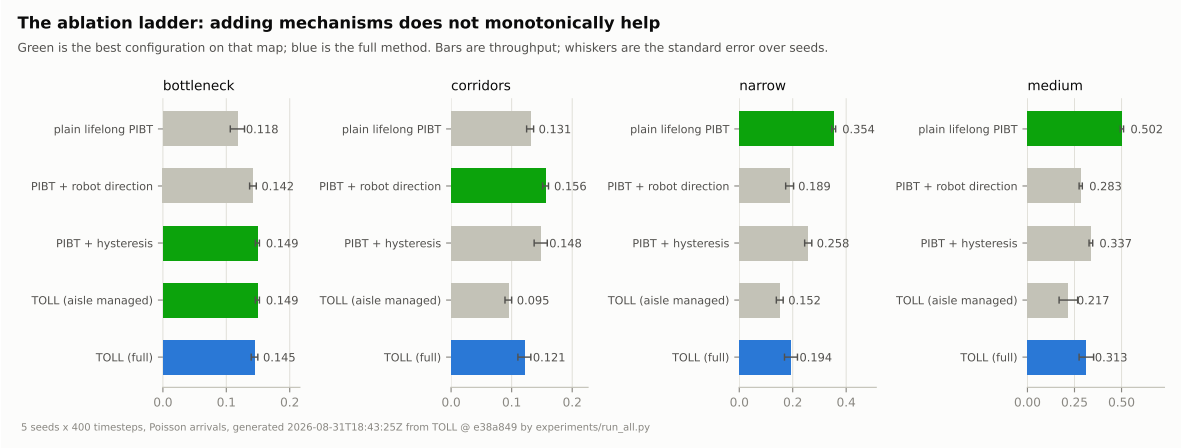

Figure 6: The cumulative ladder of Section 19.2. Adding mechanisms does not monotonically help, and on two maps the first rung is the best one.

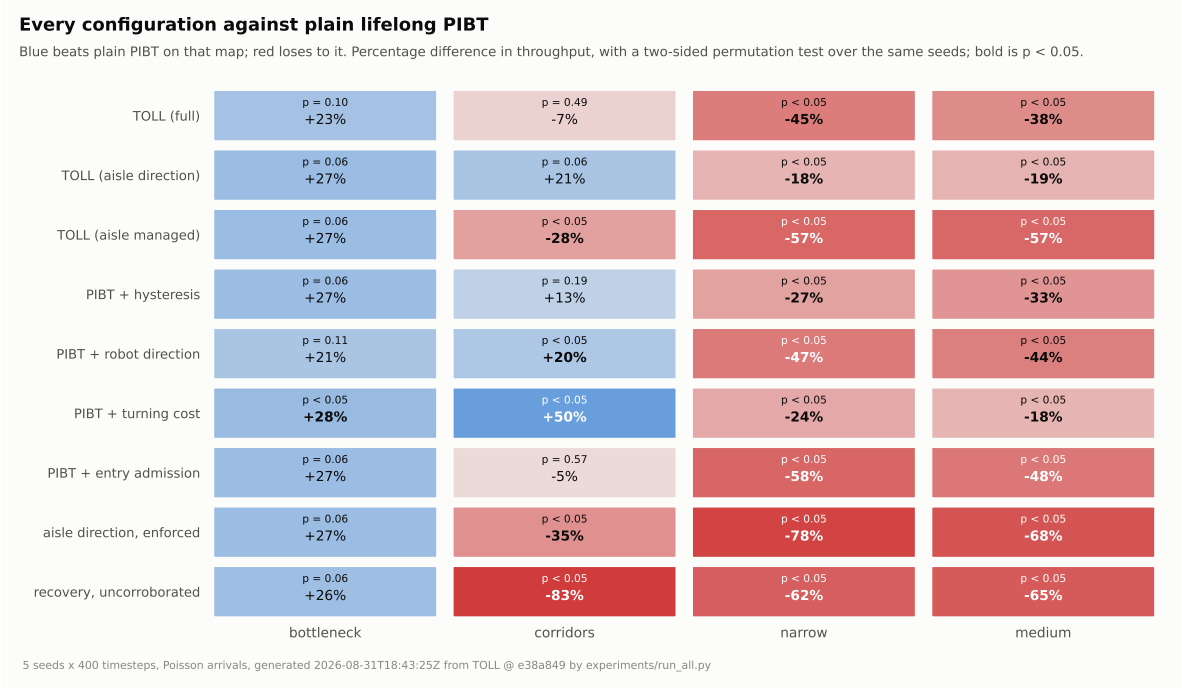

Figure 7: Every configuration against plain lifelong PIBT, with a permutation test on the same seeds.

Two rows of Figure 7 deserve comment. `turning_cost_only` — a flat anti-zigzag penalty with nothing to do with aisles — is the best variant on `warehouse_corridors` (0.196, $p = 0.008$) and the second best on `warehouse_medium`. The cheapest mechanism in the system is among the most effective, and any honest account of this work has to say so. And on `warehouse_bottleneck` every variant beats plain PIBT by 21–28% while none of them clears $p = 0.05$: with no aisle ever committing a direction, those variants differ from the base only in the turning cost, and five seeds cannot separate them.

21.6 Hypotheses

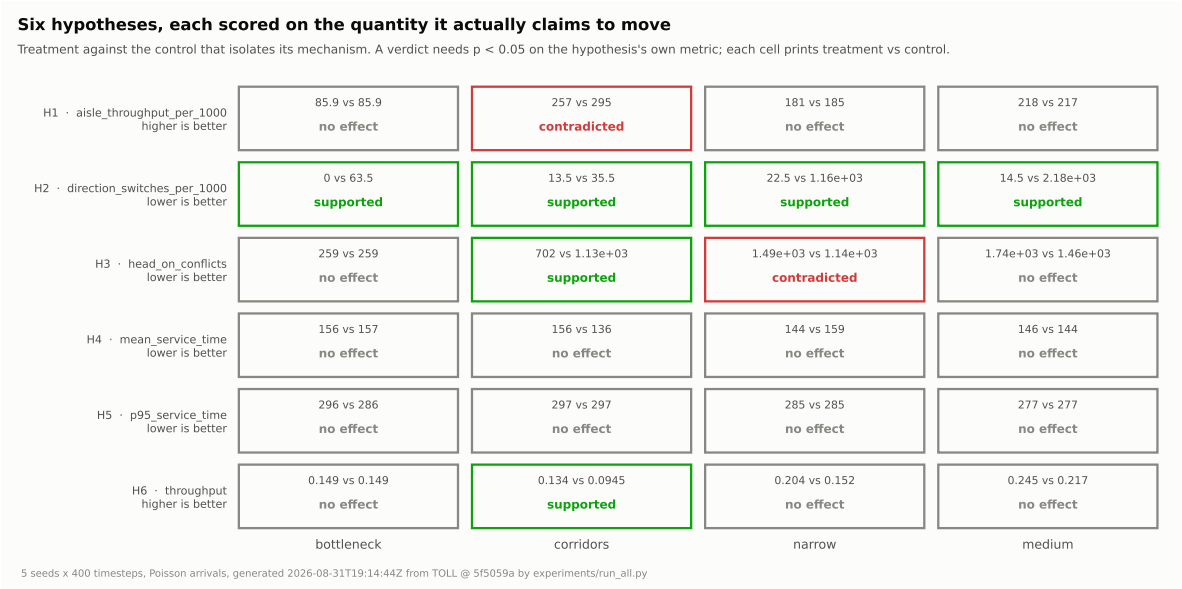

Figure 8: Each hypothesis against the control that isolates its mechanism, on the quantity it actually claims to move.

Scoring each hypothesis on its own metric rather than on global throughput is what makes these verdicts mean anything: H3 claims *fewer head-on conflicts* and H1 claims *narrow aisles flow better*, and judging either by throughput can miss a mechanism doing exactly what it promises, or credit one that is not.

- **H2 (hysteresis reduces switching) is supported on every map**, at the five-seed floor, and by two orders of magnitude: 14.5 switches per 1000 steps against 2188 on `warehouse_medium`.
- **H3 (entry admission reduces head-on conflicts) is map-dependent**: supported on corridors (702 against 1130), contradicted on narrow. Both directions are real and the map decides which.
- **H1 is contradicted on its own metric on corridors** and flat elsewhere, while the same configuration raises *global* throughput on three of four maps. Both are true and not in tension: one-way aisles route traffic through fewer aisle transits per delivered task, so per-aisle flow falls while end-to-end flow rises. The lesson is about the denominator, not the mechanism.
- **H5's mechanism is inert.** Changing R_{near} and R_{far} produces runs identical to the last decimal, because β only breaks ties among candidates with *equal* progress and the decay lowers it precisely where $\alpha\Delta$ already decides. The weight is not inert, and is net harmful: `no_direction_term` ($\beta = 0$) beats `aisle_direction_only` on medium (0.433 against 0.405) and ties it on narrow.
- **H4 and H6 are unsupported on three maps each**, and H6 is supported on corridors.

21.7 De-confounded findings

The 2×2 decomposition of Equation (55), on the design that could not be read at all before the reservation layer's gating was fixed (Section 10.5):

Map	base	aisle direction	reservations	both	interaction
corridors	0.148	**0.158**	0.125	0.095	−0.040
medium	0.337	**0.405**	0.262	0.217	−0.112

Table 8: `aisle_direction_vs_reservations`. The attribution reverses once the cells are distinct runs.

Aisle-level direction control is the flag that *helps* in isolation (+7% and +20%); entry admission is the one that costs (−16% and −22%); and the two interact strongly negatively, so the combined rung of the ladder cannot be read as one contribution plus the other. The earlier verdict — direction blamed, reservations exonerated — was reading a cell in which factor B was a bit-identical copy of the base.

21.8 The honest headline

Map	best TOLL	plain PIBT	difference	p
bottleneck (aisle-shaped)	0.149	0.118	+27%	0.056
corridors (aisle-shaped)	0.158	0.131	+21%	0.063
narrow (open banks)	0.289	0.354	−18%	0.016
medium (open grid)	0.405	0.502	−19%	0.008

Table 9: The best TOLL configuration on each map — `aisle_direction_only` in all four — against plain lifelong PIBT.

Stated plainly, and stated as the evidence actually stands:

1. **Against the published baselines, TOLL wins decisively and significantly on every map tested**, at a twentieth to a three-hundredth of their cost per timestep. So does plain lifelong PIBT. The gap between PIBT-family planners and reservation-table planners in narrow lifelong traffic is the largest effect in this entire evaluation, and it is not this project's contribution — it is PIBT's.
2. **Against plain lifelong PIBT, the aisle layer is ahead by 21–27% on the two aisle-shaped maps and behind by 18–19% on the two open ones.** The losses are significant at five seeds; *the wins are not* ($p = 0.056$ and 0.063). The direction is consistent with the design and with the mechanism, and the evidence for it is currently thin. Ten seeds would settle it either way, and that is the obvious next run rather than a caveat to argue around.
3. **The mechanisms of Section 3 are established.** Pricing direction instead of enforcing it, and requiring corroboration before escalating recovery, are large and significant wherever their mechanism can act at all. They are what took the aisle layer from catastrophic to competitive.
4. **The cheapest mechanism in the system is the one with least to do with aisles.** A flat turning cost is the best variant on corridors and second best on medium.

The value of this pass is not that every hypothesis now passes; three of six still do not. It is that H1, H3, H4 and H6 were given a *valid* test for the first time — H3's treatment cell was previously empty, H1's headline map produces no treatment, and H6's support came from rescuing deadlocks the architecture manufactured for it.

21.9 Reading these numbers

`docs/metrics.md` is the companion to this section: every reported quantity, which direction is good, and how each one misleads on its own. Three results there are worth carrying back into this document, because they are the clearest statement of what the aisle layer buys and what it costs, and none of them is visible in a raw column:

- **Travel distance per delivered task** falls 23% on `warehouse_bottleneck` ($p = 0.008$) and 25% on `warehouse_corridors` ($p = 0.016$) — the fleet drives a quarter less to deliver the same task — and rises 13% and 21% on the two open maps. The raw total is lower everywhere and means nothing: a planner that delivers less also drives less.
- **Head-on conflicts per 1000 steps** fall 34% on `warehouse_bottleneck` ($p = 0.016$), and 52% per delivered task. That is the mechanism measured on its own terms rather than through throughput.
- **Backtracks per delivered task** rise by 85–130% wherever an aisle commits a direction ($p = 0.008$). Pricing counterflow makes the inheritance recursion explore further before it resolves, and that is the largest single cost of the method.

21.10 Threats to validity

1. **Task assignment is held constant** across all five planners (Section 13). This isolates the movement layer, and it also means neither baseline was tested with the assignment policy its own paper pairs it with.
2. **RHCR's window and replan period (10/5) are untuned defaults**, not values chosen for this scale. RHCR targets hundreds of robots on much larger warehouses, where a rolling horizon pays for itself against full CBS. Its poor showing here is a property

of *this baseline at this scale with these defaults*, and is not evidence about RHCR in general.

3. **Token Passing's gridlock, by contrast, is structural** — no priority inheritance, no backtracking — and is well documented in the lifelong-MAPD literature. It will not go away with different hyperparameters.
4. **Five seeds is thin**, as Table 9 shows directly. Every claim of significance here rests on a test whose floor is $p = 0.008$, and every claim of non-significance may simply be underpowered.
5. **Four maps, one arrival process, one fleet size per map.** The boundary this document draws between "aisle-shaped" and "open" is drawn through four points.

References

Rajendra Jain, Dah-Ming Chiu, and William Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. Technical Report DEC-TR-301, Digital Equipment Corporation, 1984.

Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Addison-Wesley, 3rd edition, 1997.

Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W. Durham, T. K. Satish Kumar, and Sven Koenig. Lifelong multi-agent path finding in large-scale warehouses. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 11272–11281, 2021.

Hang Ma, Jiaoyang Li, T. K. Satish Kumar, and Sven Koenig. Lifelong multi-agent path finding for online pickup and delivery tasks. In *Proceedings of the 16th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 837–845, 2017.

Keisuke Okumura, Manao Machida, Xavier Défago, and Yasumasa Tamura. Priority inheritance with backtracking for iterative multi-agent path finding. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 535–542, 2019. arXiv:1901.11282.

Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015.